

Chapter 8. Memory Management

We saw in [Chapter 2](#) how Linux takes advantage of 80 x 86's segmentation and paging circuits to translate logical addresses into physical ones. We also mentioned that some portion of RAM is permanently assigned to the kernel and used to store both the kernel code and the static kernel data structures.

The remaining part of the RAM is called dynamic memory . It is a valuable resource, needed not only by the processes but also by the kernel itself. In fact, the performance of the entire system depends on how efficiently dynamic memory is managed. Therefore, all current multitasking operating systems try to optimize the use of dynamic memory, assigning it only when it is needed and freeing it as soon as possible. [Figure 8-1](#) shows schematically the page frames used as dynamic memory; see the section "[Physical Memory Layout](#)" in [Chapter 2](#) for details.

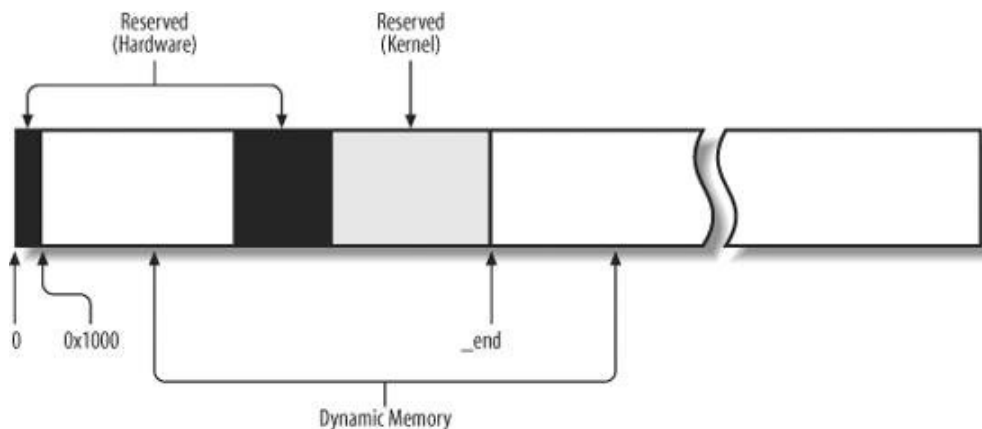
This chapter, which consists of three main sections, describes how the kernel allocates dynamic memory for its own use. The sections "[Page Frame Management](#)" and "[Memory Area Management](#)" illustrate two different techniques for handling physically contiguous memory areas, while the section "[Noncontiguous Memory Area Management](#)" illustrates a third technique that handles noncontiguous memory areas. In these sections we'll cover topics such as memory zones, kernel mappings, the buddy system, the slab cache, and memory pools.

8.1. Page Frame Management

We saw in the section "[Paging in Hardware](#)" in [Chapter 2](#) how the Intel Pentium processor can use two different page frame sizes: 4 KB and 4 MB (or 2 MB if PAE is enabledsee the section "[The Physical Address Extension \(PAE\) Paging Mechanism](#)" in [Chapter 2](#)). Linux adopts the smaller 4 KB page frame size as the standard memory allocation unit. This makes things simpler for two reasons:

- The Page Fault exceptions issued by the paging circuitry are easily interpreted. Either the page requested exists but the process is not allowed to address it, or the page does not exist. In the second case, the memory allocator must find a free 4 KB page frame and assign it to the process.
- Although both 4 KB and 4 MB are multiples of all disk block sizes, transfers of data between main memory and disks are in most cases more efficient when the smaller size is used.

Figure 8-1. Dynamic memory



8.1.1. Page Descriptors

The kernel must keep track of the current status of each page frame. For instance, it must be able to distinguish the page frames that are used to contain pages that belong to processes from those that contain kernel code or kernel data structures. Similarly, it must be able to determine whether a page frame in dynamic memory is free. A page frame in dynamic memory is free if it does not contain any useful data. It is not free when the page frame contains data of a User Mode process, data of a software cache, dynamically allocated kernel data structures, buffered data of a device driver, code of a kernel module, and so on.

State information of a page frame is kept in a page descriptor of type `page`, whose fields are shown in [Table 8-1](#). All page descriptors are stored in the `mem_map` array. Because

each descriptor is 32 bytes long, the space required by `mem_map` is slightly less than 1% of the whole RAM. The `virt_to_page(addr)` macro yields the address of the page descriptor associated with the linear address `addr`. The `pfn_to_page(pfn)` macro yields the address of the page descriptor associated with the page frame having number `pfn`.

Table 8-1. The fields of the page descriptor

Type	Name	Description
		Array of flags (see Table 8-2). Also encodes the zone number to which the page frame belongs.
<code>unsigned long</code>	<code>flags</code>	
<code>atomic_t</code>	<code>_count</code>	Page frame's reference counter.
<code>atomic_t</code>	<code>_mapcount</code>	Number of Page Table entries that refer to the page frame (−1 if none). Available to the kernel component that is using the page (for instance, it is a buffer head pointer in case of buffer page; see " Block Buffers and Buffer Heads " in Chapter 15). If the page is free, this field is used by the buddy system (see later in this chapter).
<code>unsigned long</code>	<code>private</code>	Used when the page is inserted into the page cache (see the section
<code>struct</code>	<code>mapping</code>	
<code>address_space *</code>		

unsigned long	index	"The Page Cache" in Chapter 15), or when it belongs to an anonymous region (see the section "Reverse Mapping for Anonymous Pages" in Chapter 17). Used by several kernel components with different meanings. For instance, it identifies the position of the data stored in the page frame within the page's disk image or within an anonymous region (Chapter 15), or it stores a swapped-out page identifier (Chapter 17). Contains pointers to the least recently used doubly linked list of pages.
struct list_head	lru	

You don't have to fully understand the role of all fields in the page descriptor right now. In the following chapters, we often come back to the fields of the page descriptor. Moreover, several fields have different meaning, according to whether the page frame is free or what kernel component is using the page frame.

Let's describe in greater detail two of the fields:

_count

A usage reference counter for the page. If it is set to -1, the corresponding page frame is free and can be assigned to any process or to the kernel itself. If it is set to a value greater than or equal to 0, the page frame is assigned to one or more processes or is used to store some kernel data structures. The `page_count()` function returns the value of the `_count` field increased by one, that is, the number of users of the page.

flags

Includes up to 32 flags (see [Table 8-2](#)) that describe the status of the page frame. For each `PG_xyz` flag, the kernel defines some macros that manipulate its value. Usually, the `PageXYZ` macro returns the value of the flag, while the `SetPageXYZ` and `ClearPageXYZ` macro set and clear the corresponding bit, respectively.

Table 8-2. Flags describing the status of a page frame

Flag name

Meaning

`PG_locked`

The page is locked; for instance, it is involved in a disk I/O operation.

`PG_error`

An I/O error occurred while transferring the page.

`PG_referenced`

The page has been recently accessed.

`PG_uptodate`

This flag is set after completing a read operation, unless a disk I/O error happened.

`PG_dirty`

The page has been modified (see the section "[Implementing the PFRA](#)" in [Chapter 17](#)).

`PG_lru`

The page is in the active or inactive page list (see the section "[The Least Recently Used \(LRU\) Lists](#)" in [Chapter 17](#)).

`PG_active`

The page is in the active page list (see the section "[The Least Recently Used \(LRU\) Lists](#)" in [Chapter 17](#)).

`PG_slab`

The page frame is included in a slab (see the section "[Memory Area Management](#)" later in this chapter).

`PG_highmem`

The page frame belongs to the `ZONE_HIGHMEM` zone (see the following section "[Non-Uniform Memory Access \(NUMA\)](#)").

`PG_checked`

Used by some filesystems such as Ext2 and Ext3 (see [Chapter 18](#)).

`PG_arch_1`

Not used on the 80 x 86 architecture.

`PG_reserved`

The page frame is reserved for kernel code or is unusable.

`PG_private`

The `private` field of the page descriptor stores meaningful data.

`PG_writeback`

The page is being written to disk by means of the `writepage` method (see [Chapter 16](#)).

`PG_nosave`

Used for system suspend/resume.

`PG_compound`

The page frame is handled through the extended paging mechanism (see the section "[Extended Paging](#)" in [Chapter 2](#)).

`PG_swapcache`

The page belongs to the swap cache (see the section "[The Swap Cache](#)" in [Chapter 17](#)).

`PG_mappedtodisk`

All data in the page frame corresponds to blocks allocated on disk.

`PG_reclaim`

The page has been marked to be written to disk in order to reclaim memory.

`PG_nosave_free`

Used for system suspend/resume.

8.1.2. Non-Uniform Memory Access (NUMA)

We are used to thinking of the computer's memory as a homogeneous, shared resource. Disregarding the role of the hardware caches, we expect the time required for a CPU to access a memory location to be essentially the same, regardless of the location's physical address and the CPU. Unfortunately, this assumption is not true in some architectures. For instance, it is not true for some multiprocessor Alpha or MIPS computers.

Linux 2.6 supports the Non-Uniform Memory Access (NUMA) model, in which the access times for different memory locations from a given CPU may vary. The physical memory of the system is partitioned in several nodes. The time needed by a given CPU to access pages within a single node is the same. However, this time might not be the same for two different CPUs. For every CPU, the kernel tries to minimize the number of accesses to costly nodes by carefully selecting where the kernel data structures that are most often referenced by the CPU are stored.^[*]

[*] Furthermore, the Linux kernel makes use of NUMA even for some peculiar uniprocessor systems that have huge "holes" in the physical address space. The kernel handles these architectures by assigning the contiguous subranges of valid physical addresses to different memory nodes.

The physical memory inside each node can be split into several zones, as we will see in the next section. Each node has a descriptor of type `pg_data_t`, whose fields are shown in [Table 8-3](#). All node descriptors are stored in a singly linked list, whose first element is pointed to by the `pgdat_list` variable.

Table 8-3. The fields of the node descriptor

Type

Name

Description

```
struct zone [ ]
```

```
node_zones
```

Array of zone descriptors of the node

```
struct zonelist [ ]
```

```
node_zonelists
```

Array of `zonelist` data structures used by the page allocator (see the later section "[Memory Zones](#)")

```
int
```

```
nr_zones
```

Number of zones in the node

```
struct page *
```

```
node_mem_map
```

Array of page descriptors of the node

```
struct
```

```
bootmem_data *
```

```
bdata
```

Used in the kernel initialization phase

```
unsigned long
```

```
node_start_pfn
```

Index of the first page frame in the node

```
unsigned long
```

```
node_present_pages
```

Size of the memory node, excluding holes (in page frames)

```
unsigned long
```

```
node_spanned_pages
```

Size of the node, including holes (in page frames)

```
int
```

```
node_id
```

Identifier of the node

```
pg_data_t *
```

```
pgdat_next
```

Next item in the memory node list

```
wait_queue_head_t
```

```
kswapd_wait
```

Wait queue for the kswapd pageout daemon (see the section "[Periodic Reclaiming](#)" in [Chapter 17](#))

```
struct task_struct *
```

```
kswapd
```

Pointer to the process descriptor of the kswapd kernel thread

```
int
```

kswapd_max_order

Logarithmic size of free blocks to be created by kswapd

As usual, we are mostly concerned with the 80 x 86 architecture. IBM-compatible PCs use the Uniform Memory Access model (UMA), thus the NUMA support is not really required. However, even if NUMA support is not compiled in the kernel, Linux makes use of a single node that includes all system physical memory. Thus, the `pgdat_list` variable points to a list consisting of a single element the node 0 descriptor stored in the `contig_page_data` variable.

On the 80 x 86 architecture, grouping the physical memory in a single node might appear useless; however, this approach makes the memory handling code more portable, because the kernel can assume that the physical memory is partitioned in one or more nodes in all architectures.^[*]

[*] We have another example of this kind of design choice: Linux uses four levels of Page Tables even when the hardware architecture defines just two levels (see the section "[Paging in Linux](#)" in [Chapter 2](#)).

8.1.3. Memory Zones

In an ideal computer architecture, a page frame is a memory storage unit that can be used for anything: storing kernel and user data, buffering disk data, and so on. Every kind of page of data can be stored in a page frame, without limitations.

However, real computer architectures have hardware constraints that may limit the way page frames can be used. In particular, the Linux kernel must deal with two hardware constraints of the 80 x 86 architecture:

- The Direct Memory Access (DMA) processors for old ISA buses have a strong limitation: they are able to address only the first 16 MB of RAM.
- In modern 32-bit computers with lots of RAM, the CPU cannot directly access all physical memory because the linear address space is too small.

To cope with these two limitations, Linux 2.6 partitions the physical memory of every memory node into three zones. In the 80 x 86 UMA architecture the zones are:

ZONE_DMA

Contains page frames of memory below 16 MB

ZONE_NORMAL

Contains page frames of memory at and above 16 MB and below 896 MB

ZONE_HIGHMEM

Contains page frames of memory at and above 896 MB

The `ZONE_DMA` zone includes page frames that can be used by old ISA-based devices by means of the DMA.

(The section "[Direct Memory Access \(DMA\)](#)" in [Chapter 13](#) gives further details on DMA.)

The `ZONE_DMA` and `ZONE_NORMAL` zones include the "normal" page frames that can be directly accessed by the kernel through the linear mapping in the fourth gigabyte of the linear address space (see the section "[Kernel Page Tables](#)" in [Chapter 2](#)). Conversely, the `ZONE_HIGHMEM` zone includes page frames that cannot be directly accessed by the kernel through the linear mapping in the fourth gigabyte of linear address space (see the section "[Kernel Mappings of High-Memory Page Frames](#)" later in this chapter). The `ZONE_HIGHMEM` zone is always empty on 64-bit architectures.

Each memory zone has its own descriptor of type `zone`. Its fields are shown in [Table 8-4](#).

Table 8-4. The fields of the zone descriptor

Type

Name

Description

`unsigned long`

`free_pages`

Number of free pages in the zone.

`unsigned long`

`pages_min`

Number of reserved pages of the zone (see the section "[The Pool of Reserved Page Frames](#)" later in this chapter).

`unsigned long`

`pages_low`

Low watermark for page frame reclaiming; also used by the zone allocator as a threshold value (see the section "[The Zone Allocator](#)" later in this chapter).

`unsigned long`

`pages_high`

High watermark for page frame reclaiming; also used by the zone allocator as a threshold value.

`unsigned long []`

`lowmem_reserve`

Specifies how many page frames in each zone must be reserved for handling low-on-memory critical situations.

`struct per_cpu_pageset[]`

pageset

Data structure used to implement special caches of single page frames (see the section "[The Per-CPU Page Frame Cache](#)" later in this chapter).

spinlock_t

lock

Spin lock protecting the descriptor.

struct free_area []

free_area

Identifies the blocks of free page frames in the zone (see the section "[The Buddy System Algorithm](#)" later in this chapter).

spinlock_t

lru_lock

Spin lock for the active and inactive lists.

struct list head

active_list

List of active pages in the zone (see [Chapter 17](#)).

struct list head

inactive_list

List of inactive pages in the zone (see [Chapter 17](#)).

unsigned long

nr_scan_active

Number of active pages to be scanned when reclaiming memory (see the section "[Low On Memory Reclaiming](#)" in [Chapter 17](#)).

unsigned long

nr_scan_inactive

Number of inactive pages to be scanned when reclaiming memory.

unsigned long

nr_active

Number of pages in the zone's active list.

unsigned long

nr_inactive

Number of pages in the zone's inactive list.

unsigned long

pages_scanned

Counter used when doing page frame reclaiming in the zone.

int

all_unreclaimable

Flag set when the zone is full of unreclaimable pages.

int

temp_priority

Temporary zone's priority (used when doing page frame reclaiming).

int

prev_priority

Zone's priority ranging between 12 and 0 (used by the page frame reclaiming algorithm, see the section "[Low On Memory Reclaiming](#)" in [Chapter 17](#)).

wait_queue_head_t *

wait_table

Hash table of wait queues of processes waiting for one of the pages of the zone.

unsigned long

wait_table_size

Size of the wait queue hash table.

unsigned long

wait_table_bits

Power-of-2 order of the size of the wait queue hash table array.

struct pglist_data *

zone_pgdat

Memory node (see the earlier section "[Non-Uniform Memory Access \(NUMA\)](#)").

```
struct page *
```

```
zone_mem_map
```

Pointer to first page descriptor of the zone.

```
unsigned long
```

```
zone_start_pfn
```

Index of the first page frame of the zone.

```
unsigned long
```

```
spanned_pages
```

Total size of zone in pages, including holes.

```
unsigned long
```

```
present_pages
```

Total size of zone in pages, excluding holes.

```
char *
```

```
name
```

Pointer to the conventional name of the zone: "DMA," "Normal," or "HighMem."

Many fields of the `zone` structure are used for page frame reclaiming and will be described in [Chapter 17](#).

Each page descriptor has links to the memory node and to the zone inside the node that includes the corresponding page frame. To save space, these links are not stored as classical pointers; rather, they are encoded as indices stored in the high bits of the `flags` field. In fact, the number of flags that characterize a page frame is limited, thus it is always possible to reserve the most significant bits of the `flags` field to encode the proper memory node and zone number.^[*] The `page_zone( )` function receives as its parameter the address of a page descriptor; it reads the most significant bits of the `flags` field in the page descriptor, then it determines the address of the corresponding zone descriptor by looking in the `zone_table` array. This array is initialized at boot time with the addresses of all zone descriptors of all memory nodes.

[*] The number of bits reserved for the indices depends on whether the kernel supports the NUMA model and on the size of the `flags` field. If NUMA is not supported, the `flags` field has two bits for the zone index and one bit always set to zero for the node index. On NUMA 32-bit architectures, `flags` has two bits for the zone index and six bits for the node number. Finally, on NUMA 64-bit architectures, the 64-bit `flags` field has 2 bits for the zone index and 10 bits for the node number.

When the kernel invokes a memory allocation function, it must specify the zones that contain the requested page frames. The kernel usually specifies which zones it's willing to use. For instance, if a page frame must be directly mapped in the fourth gigabyte of linear addresses but it is not going to be used for ISA DMA transfers, then the kernel requests a page frame either in `ZONE_NORMAL` or in `ZONE_DMA`. Of course, the

page frame should be obtained from `ZONE_DMA` only if `ZONE_NORMAL` does not have free page frames. To specify the preferred zones in a memory allocation request, the kernel uses the `zonelist` data structure, which is an array of zone descriptor pointers.

8.1.4. The Pool of Reserved Page Frames

Memory allocation requests can be satisfied in two different ways. If enough free memory is available, the request can be satisfied immediately. Otherwise, some memory reclaiming must take place, and the kernel control path that made the request is blocked until additional memory has been freed.

However, some kernel control paths cannot be blocked while requesting memory; this happens, for instance, when handling an interrupt or when executing code inside a critical region. In these cases, a kernel control path should issue atomic memory allocation requests (using the `GFP_ATOMIC` flag; see the later section "[The Zoned Page Frame Allocator](#)"). An atomic request never blocks: if there are not enough free pages, the allocation simply fails.

Although there is no way to ensure that an atomic memory allocation request never fails, the kernel tries hard to minimize the likelihood of this unfortunate event. In order to do this, the kernel reserves a pool of page frames for atomic memory allocation requests to be used only on low-on-memory conditions.

The amount of the reserved memory (in kilobytes) is stored in the `min_free_kbytes` variable. Its initial value is set during kernel initialization and depends on the amount of physical memory that is directly mapped in the kernel's fourth gigabyte of linear address; that is, it depends on the number of page frames included in the `ZONE_DMA` and `ZONE_NORMAL` memory zones:

$$\text{reserved pool size} = \lfloor \sqrt{16 \times \text{directly mapped memory}} \rfloor \text{ (kilobytes)}$$

However, initially `min_free_kbytes` cannot be lower than 128 and greater than 65,536.^[*]

[*] The amount of reserved memory can be changed later by the system administrator either by writing in the `/proc/sys/vm/min_free_kbytes` file or by issuing a suitable `sysctl( )` system call.

The `ZONE_DMA` and `ZONE_NORMAL` memory zones contribute to the reserved memory with a number of page frames proportional to their relative sizes. For instance, if the `ZONE_NORMAL` zone is eight times bigger than `ZONE_DMA`, seven-eighths of the page frames will be taken from `ZONE_NORMAL` and one-eighth from `ZONE_DMA`.

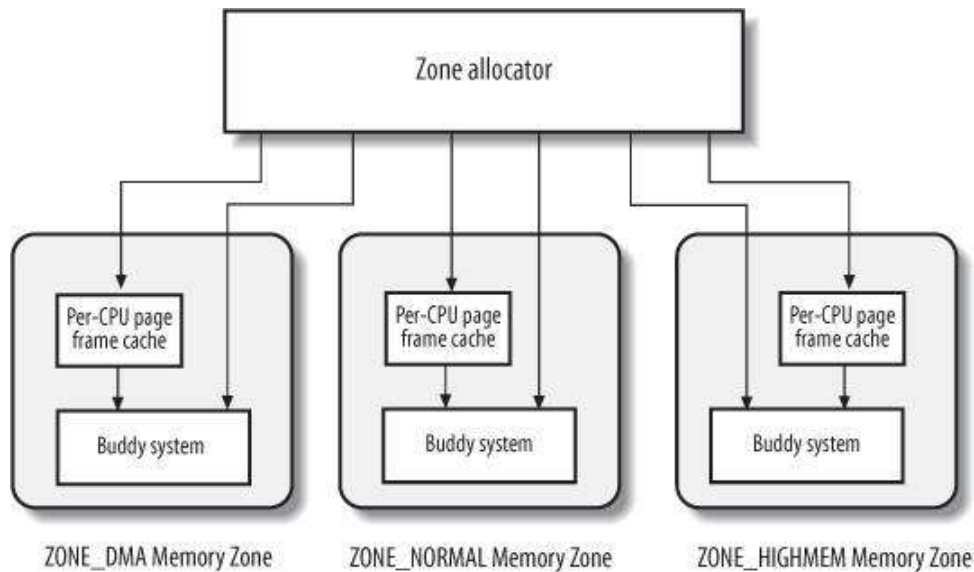
The `pages_min` field of the zone descriptor stores the number of reserved page frames inside the zone. As we'll see in [Chapter 17](#), this field plays also a role for the page frame reclaiming algorithm, together with the `pages_low` and `pages_high` fields. The `pages_low` field is always set to 5/4 of the value of `pages_min`, and `pages_high` is always set to 3/2 of the value of `pages_min`.

8.1.5. The Zoned Page Frame Allocator

The kernel subsystem that handles the memory allocation requests for groups of contiguous page frames is called the zoned page frame allocator. Its main components are shown in [Figure 8-2](#).

The component named "zone allocator " receives the requests for allocation and deallocation of dynamic memory. In the case of allocation requests, the component searches a memory zone that includes a group of contiguous page frames that can satisfy the request (see the later section "[The Zone Allocator](#)"). Inside each zone, page frames are handled by a component named "buddy system " (see the later section "[The Buddy System Algorithm](#)"). To get better system performance, a small number of page frames are kept in cache to quickly satisfy the allocation requests for single page frames (see the later section "[The Per-CPU Page Frame Cache](#)").

Figure 8-2. Components of the zoned page frame allocator



8.1.5.1. Requesting and releasing page frames

Page frames can be requested by using six slightly different functions and macros. Unless otherwise stated, they return the linear address of the first allocated page or return `NULL` if the allocation failed.

`alloc_pages(gfp_mask, order)`

Macro used to request 2^{order} contiguous page frames. It returns the address of the descriptor of the first allocated page frame or returns `NULL` if the allocation failed.

`alloc_page(gfp_mask)`

Macro used to get a single page frame; it expands to:

```
alloc_pages(gfp_mask, 0)
```

It returns the address of the descriptor of the allocated page frame or returns `NULL` if the allocation failed.

`__get_free_pages(gfp_mask, order)`

Function that is similar to `alloc_pages( )`, but it returns the linear address of the first allocated page.

`__get_free_page(gfp_mask)`

Macro used to get a single page frame; it expands to:

```
__get_free_pages(gfp_mask, 0)
```

`get_zeroed_page(gfp_mask)`

Function used to obtain a page frame filled with zeros; it invokes:

```
alloc_pages(gfp_mask | __GFP_ZERO, 0)
```

and returns the linear address of the obtained page frame.

`__get_dma_pages(gfp_mask, order)`

Macro used to get page frames suitable for DMA; it expands to:

```
__get_free_pages(gfp_mask | __GFP_DMA, order)
```

The parameter `gfp_mask` is a group of flags that specify how to look for free page frames. The flags that can be used in `gfp_mask` are shown in [Table 8-5](#).

Table 8-5. Flag used to request page frames

Flag

Description

`__GFP_DMA`

The page frame must belong to the `ZONE_DMA` memory zone. Equivalent to `GFP_DMA`.

`__GFP_HIGHMEM`

The page frame may belong to the `ZONE_HIGHMEM` memory zone.

`__GFP_WAIT`

The kernel is allowed to block the current process waiting for free page frames.

`__GFP_HIGH`

The kernel is allowed to access the pool of reserved page frames.

`__GFP_IO`

The kernel is allowed to perform I/O transfers on low memory pages in order to free page frames.

`__GFP_FS`

If clear, the kernel is not allowed to perform filesystem-dependent operations.

`__GFP_COLD`

The requested page frames may be "cold" (see the later section "[The Per-CPU Page Frame Cache](#)").

`__GFP_NOWARN`

A memory allocation failure will not produce a warning message.

`__GFP_REPEAT`

The kernel keeps retrying the memory allocation until it succeeds.

`__GFP_NOFAIL`

Same as `__GFP_REPEAT`.

`__GFP_NORETRY`

Do not retry a failed memory allocation.

`__GFP_NO_GROW`

The slab allocator does not allow a slab cache to be enlarged (see the later section "[The Slab Allocator](#)").

`__GFP_COMP`

The page frame belongs to an extended page (see the section "[Extended Paging](#)" in [Chapter 2](#)).

`__GFP_ZERO`

The page frame returned, if any, must be filled with zeros.

In practice, Linux uses the predefined combinations of flag values shown in [Table 8-6](#); the group name is what you'll encounter as the argument of the six page frame allocation functions.

Table 8-6. Groups of flag values used to request page frames

Group name

Corresponding flags

`GFP_ATOMIC`

```

__ _GFP_HIGH

GFP_NOIO

__ _GFP_WAIT

GFP_NOFS

__ _GFP_WAIT | __ _GFP_IO

GFP_KERNEL

__ _GFP_WAIT | __ _GFP_IO | __ _GFP_FS

GFP_USER

__ _GFP_WAIT | __ _GFP_IO | __ _GFP_FS

GFP_HIGHUSER

__ _GFP_WAIT | __ _GFP_IO | __ _GFP_FS | __ _GFP_HIGHMEM

```

The `__ _GFP_DMA` and `__ _GFP_HIGHMEM` flags are called zone modifiers ; they specify the zones searched by the kernel while looking for free page frames. The `node_zonelists` field of the `contig_page_data` node descriptor is an array of lists of zone descriptors representing the fallback zones: for each setting of the zone modifiers, the corresponding list includes the memory zones that could be used to satisfy the memory allocation request in case the original zone is short on page frames. In the 80 x 86 UMA architecture, the fallback zones are the following:

- If the `__ _GFP_DMA` flag is set, page frames can be taken only from the `ZONE_DMA` memory zone.
- Otherwise, if the `__ _GFP_HIGHMEM` flag is not set, page frames can be taken only from the `ZONE_NORMAL` and the `ZONE_DMA` memory zones, in order of preference.
- Otherwise (the `__ _GFP_HIGHMEM` flag is set), page frames can be taken from `ZONE_HIGHMEM`, `ZONE_NORMAL`, and `ZONE_DMA` memory zones, in order of preference.

Page frames can be released through each of the following four functions and macros:

`__ _free_pages(page, order)`

This function checks the page descriptor pointed to by `page`; if the page frame is not reserved (i.e., if the `PG_reserved` flag is equal to 0), it decreases the `count` field of the descriptor. If `count` becomes 0, it assumes that 2^{order} contiguous page frames starting from the one corresponding to `page` are no longer used. In this case, the function releases the page frames as explained in the later section "[The Zone Allocator](#)."

`free_pages(addr, order)`

This function is similar to `__ _free_pages( )`, but it receives as an argument the linear address `addr` of the first page frame to be released.

`__free_page(page)`

This macro releases the page frame having the descriptor pointed to by `page`; it expands to:

```
__free_pages(page, 0)
```

`free_page(addr)`

This macro releases the page frame having the linear address `addr`; it expands to:

```
free_pages(addr, 0)
```

8.1.6. Kernel Mappings of High-Memory Page Frames

The linear address that corresponds to the end of the directly mapped physical memory, and thus to the beginning of the high memory, is stored in the `high_memory` variable, which is set to 896 MB. Page frames above the 896 MB boundary are not generally mapped in the fourth gigabyte of the kernel linear address spaces, so the kernel is unable to directly access them. This implies that each page allocator function that returns the linear address of the assigned page frame doesn't work for high-memory page frames, that is, for page frames in the `ZONE_HIGHMEM` memory zone.

For instance, suppose that the kernel invoked `__get_free_pages(GFP_HIGHMEM, 0)` to allocate a page frame in high memory. If the allocator assigned a page frame in high memory, `__get_free_pages()` cannot return its linear address because it doesn't exist; thus, the function returns `NULL`. In turn, the kernel cannot use the page frame; even worse, the page frame cannot be released because the kernel has lost track of it.

This problem does not exist on 64-bit hardware platforms, because the available linear address space is much larger than the amount of RAM that can be installed. In short, the `ZONE_HIGHMEM` zone of these architectures is always empty. On 32-bit platforms such as the 80 x 86 architecture, however, Linux designers had to find some way to allow the kernel to exploit all the available RAM, up to the 64 GB supported by PAE. The approach adopted is the following:

- The allocation of high-memory page frames is done only through the `alloc_pages()` function and its `alloc_page()` shortcut. These functions do not return the linear address of the first allocated page frame, because if the page frame belongs to the high memory, such linear address simply does not exist. Instead, the functions return the linear address of the page descriptor of the first allocated page frame. These linear addresses always exist, because all page descriptors are allocated in low memory once and forever during the kernel initialization.
- Page frames in high memory that do not have a linear address cannot be accessed by the kernel. Therefore, part of the last 128 MB of the kernel linear address space is dedicated to mapping high-memory page frames. Of course, this kind of mapping is temporary, otherwise only 128 MB of high memory would be accessible. Instead, by recycling linear addresses the whole high memory can be accessed, although at different times.

The kernel uses three different mechanisms to map page frames in high memory; they are called permanent kernel mapping, temporary kernel mapping, and noncontiguous memory allocation. In this section, we'll cover the first two techniques; the third one is discussed in the section "[Noncontiguous Memory Area Management](#)" later in this chapter.

Establishing a permanent kernel mapping may block the current process; this happens when no free Page Table entries exist that can be used as "windows" on the page frames in high memory. Thus, a permanent kernel mapping cannot be established in interrupt handlers and deferrable functions. Conversely, establishing a temporary kernel mapping never requires blocking the current process; its drawback, however, is that very few temporary kernel mappings can be established at the same time.

A kernel control path that uses a temporary kernel mapping must ensure that no other kernel control path is using the same mapping. This implies that the kernel control path can never block, otherwise another kernel control path might use the same window to map some other high memory page.

Of course, none of these techniques allow addressing the whole RAM simultaneously. After all, less than 128 MB of linear address space are left for mapping the high memory, while PAE supports systems having up to 64 GB of RAM.

8.1.6.1. Permanent kernel mappings

Permanent kernel mappings allow the kernel to establish long-lasting mappings of high-memory page frames into the kernel address space. They use a dedicated Page Table in the master kernel page tables. The `pkmap_page_table` variable stores the address of this Page Table, while the `LAST_PKMAP` macro yields the number of entries. As usual, the Page Table includes either 512 or 1,024 entries, according to whether PAE is enabled or disabled (see the section "[The Physical Address Extension \(PAE\) Paging Mechanism](#)" in [Chapter 2](#)); thus, the kernel can access at most 2 or 4 MB of high memory at once.

The Page Table maps the linear addresses starting from `PKMAP_BASE`. The `pkmap_count` array includes `LAST_PKMAP` counters, one for each entry of the `pkmap_page_table` Page Table. We distinguish three cases:

The counter is 0

The corresponding Page Table entry does not map any high-memory page frame and is usable.

The counter is 1

The corresponding Page Table entry does not map any high-memory page frame, but it cannot be used because the corresponding TLB entry has not been flushed since its last usage.

The counter is n (greater than 1)

The corresponding Page Table entry maps a high-memory page frame, which is used by exactly $n - 1$ kernel components.

To keep track of the association between high memory page frames and linear addresses induced by permanent kernel mappings, the kernel makes use of the `page_address_htable` hash table. This table contains one `page_address_map` data structure for each page frame in high memory that is currently mapped. In turn, this data structure contains a pointer to the page descriptor and the linear address assigned to the page frame.

The `page_address( )` function returns the linear address associated with the page frame, or `NULL` if the page frame is in high memory and is not mapped. This function, which receives as its parameter a page descriptor pointer `page`, distinguishes two cases:

1. If the page frame is not in high memory (PG_highmem flag clear), the linear address always exists and is obtained by computing the page frame index, converting it into a physical address, and finally deriving the linear address corresponding to the physical address. This is accomplished by the following code:

```
_ _va((unsigned long)(page - mem_map) << 12)
```

2. If the page frame is in high memory (PG_highmem flag set), the function looks into the page_address_htable hash table. If the page frame is found in the hash table, page_address() returns its linear address, otherwise it returns NULL.

The kmap() function establishes a permanent kernel mapping. It is essentially equivalent to the following code:

```
void * kmap(struct page * page)
{
    if (!PageHighMem(page))
        return page_address(page);
    return kmap_high(page);
}
```

The kmap_high() function is invoked if the page frame really belongs to high memory. The function is essentially equivalent to the following code:

```
void * kmap_high(struct page * page)
{
    unsigned long vaddr;
    spin_lock(&kmap_lock);
    vaddr = (unsigned long) page_address(page);
    if (!vaddr)
        vaddr = map_new_virtual(page);
    pkmap_count[(vaddr-PKMAP_BASE) >> PAGE_SHIFT]++;
    spin_unlock(&kmap_lock);
    return (void *) vaddr;
}
```

The function gets the kmap_lock spin lock to protect the Page Table against concurrent accesses in multiprocessor systems. Notice that there is no need to disable the interrupts, because kmap() cannot be invoked by interrupt handlers and deferrable functions. Next, the kmap_high() function checks whether the page frame is already mapped by invoking page_address(). If not, the function invokes map_new_virtual() to insert the page frame physical address into an entry of pkmap_page_table and to add an element to the page_address_htable hash table. Then kmap_high() increases the counter corresponding to the linear address of the page frame to take into account the new kernel component that invoked this function. Finally, kmap_high() releases the kmap_lock spin lock and returns the linear address that maps the page frame.

The map_new_virtual() function essentially executes two nested loops:

```
for (;;) {
    int count;
    DECLARE_WAITQUEUE(wait, current);
    for (count = LAST_PKMAP; count > 0; --count) {
        last_pkmap_nr = (last_pkmap_nr + 1) & (LAST_PKMAP - 1);
        if (!last_pkmap_nr) {
```

```

        flush_all_zero_pkmaps( );
        count = LAST_PKMAP;
    }
    if (!pkmap_count[last_pkmap_nr]) {
        unsigned long vaddr = PKMAP_BASE +
                               (last_pkmap_nr << PAGE_SHIFT);
        set_pte(&(pkmap_page_table[last_pkmap_nr]),
                mk_pte(page, _pgprot(0x63)));
        pkmap_count[last_pkmap_nr] = 1;
        set_page_address(page, (void *) vaddr);
        return vaddr;
    }
}
current->state = TASK_UNINTERRUPTIBLE;
add_wait_queue(&pkmap_map_wait, &wait);
spin_unlock(&kmap_lock);
schedule( );
remove_wait_queue(&pkmap_map_wait, &wait);
spin_lock(&kmap_lock);
if (page_address(page))
    return (unsigned long) page_address(page);
}

```

In the inner loop, the function scans all counters in `pkmap_count` until it finds a null value. The large `if` block runs when an unused entry is found in `pkmap_count`. That block determines the linear address corresponding to the entry, creates an entry for it in the `pkmap_page_table` Page Table, sets the count to 1 because the entry is now used, invokes `set_page_address( )` to insert a new element in the `page_address_htable` hash table, and returns the linear address.

The function starts where it left off last time, cycling through the `pkmap_count` array. It does this by preserving in a variable named `last_pkmap_nr` the index of the last used entry in the `pkmap_page_table` Page Table. Thus, the search starts from where it was left in the last invocation of the `map_new_virtual( )` function.

When the last counter in `pkmap_count` is reached, the search restarts from the counter at index 0. Before continuing, however, `map_new_virtual( )` invokes the `flush_all_zero_pkmaps( )` function, which starts another scan of the counters, looking for those that have the value 1. Each counter that has a value of 1 denotes an entry in `pkmap_page_table` that is free but cannot be used because the corresponding TLB entry has not yet been flushed. `flush_all_zero_pkmaps( )` resets their counters to zero, deletes the corresponding elements from the `page_address_htable` hash table, and issues TLB flushes on all entries of `pkmap_page_table`.

If the inner loop cannot find a null counter in `pkmap_count`, the `map_new_virtual( )` function blocks the current process until some other process releases an entry of the `pkmap_page_table` Page Table. This is achieved by inserting `current` in the `pkmap_map_wait` wait queue, setting the current state to `TASK_UNINTERRUPTIBLE`, and invoking `schedule( )` to relinquish the CPU. Once the process is awakened, the function checks whether another process has mapped the page by invoking `page_address( )`; if no other process has mapped the page yet, the inner loop is restarted.

The `kunmap( )` function destroys a permanent kernel mapping established previously by `kmap( )`. If the page is really in the high memory zone, it invokes the `kunmap_high( )` function, which is essentially equivalent to the following code:

```

void kunmap_high(struct page * page)
{
    spin_lock(&kmap_lock);
    if ((--pkmap_count[((unsigned long)page_address(page)
                        -PKMAP_BASE)>>PAGE_SHIFT]) == 1)

```

```

        if (waitqueue_active(&pkmap_map_wait))
            wake_up(&pkmap_map_wait);
    spin_unlock(&kmap_lock);
}

```

The expression within the brackets computes the index into the `pkmap_count` array from the page's linear address. The counter is decreased and compared to 1. A successful comparison indicates that no process is using the page. The function can finally wake up processes in the wait queue filled by `map_new_virtual()`, if any.

8.1.6.2. Temporary kernel mappings

Temporary kernel mappings are simpler to implement than permanent kernel mappings; moreover, they can be used inside interrupt handlers and deferrable functions, because requesting a temporary kernel mapping never blocks the current process.

Every page frame in high memory can be mapped through a window in the kernel address space namely, a Page Table entry that is reserved for this purpose. The number of windows reserved for temporary kernel mappings is quite small.

Each CPU has its own set of 13 windows, represented by the `enum km_type` data structure. Each symbol defined in this data structure such as `KM_BOUNCE_READ`, `KM_USER0`, or `KM_PTE0` identifies the linear address of a window.

The kernel must ensure that the same window is never used by two kernel control paths at the same time. Thus, each symbol in the `km_type` structure is dedicated to one kernel component and is named after the component. The last symbol, `KM_TYPE_NR`, does not represent a linear address by itself, but yields the number of different windows usable by every CPU.

Each symbol in `km_type`, except the last one, is an index of a fix-mapped linear address (see the section "[Fix-Mapped Linear Addresses](#)" in [Chapter 2](#)). The `enum fixed_addresses` data structure includes the symbols `FIX_KMAP_BEGIN` and `FIX_KMAP_END`; the latter is assigned to the index `FIX_KMAP_BEGIN + (KM_TYPE_NR * NR_CPUS) - 1`. In this manner, there are `KM_TYPE_NR` fix-mapped linear addresses for each CPU in the system. Furthermore, the kernel initializes the `kmap_pte` variable with the address of the Page Table entry corresponding to the `fix_to_virt(FIX_KMAP_BEGIN)` linear address.

To establish a temporary kernel mapping, the kernel invokes the `kmap_atomic()` function, which is essentially equivalent to the following code:

```

void * kmap_atomic(struct page * page, enum km_type type)
{
    enum fixed_addresses idx;
    unsigned long vaddr;

    current_thread_info()->preempt_count++;
    if (!PageHighMem(page))
        return page_address(page);
    idx = type + KM_TYPE_NR * smp_processor_id();
    vaddr = fix_to_virt(FIX_KMAP_BEGIN + idx);
    set_pte(kmap_pte+idx, mk_pte(page, 0x063));
    __flush_tlb_single(vaddr);
    return (void *) vaddr;
}

```

The `type` argument and the CPU identifier retrieved through `smp_processor_id( )` specify what fix-mapped linear address has to be used to map the request page. The function returns the linear address of the page frame if it doesn't belong to high memory; otherwise, it sets up the Page Table entry corresponding to the fix-mapped linear address with the page's physical address and the bits `Present`, `Accessed`, `Read/Write`, and `Dirty`. Finally, the function flushes the proper TLB entry and returns the linear address.

To destroy a temporary kernel mapping, the kernel uses the `kunmap_atomic( )` function. In the 80 x 86 architecture, this function decreases the `preempt_count` of the current process; thus, if the kernel control path was preemptable right before requiring a temporary kernel mapping, it will be preemptable again after it has destroyed the same mapping. Moreover, `kunmap_atomic( )` checks whether the `TIF_NEED_RESCHED` flag of `current` is set and, if so, invokes `schedule( )`.

8.1.7. The Buddy System Algorithm

The kernel must establish a robust and efficient strategy for allocating groups of contiguous page frames. In doing so, it must deal with a well-known memory management problem called external fragmentation: frequent requests and releases of groups of contiguous page frames of different sizes may lead to a situation in which several small blocks of free page frames are "scattered" inside blocks of allocated page frames. As a result, it may become impossible to allocate a large block of contiguous page frames, even if there are enough free pages to satisfy the request.

There are essentially two ways to avoid external fragmentation:

- Use the paging circuitry to map groups of noncontiguous free page frames into intervals of contiguous linear addresses.
- Develop a suitable technique to keep track of the existing blocks of free contiguous page frames, avoiding as much as possible the need to split up a large free block to satisfy a request for a smaller one.

The second approach is preferred by the kernel for three good reasons:

- In some cases, contiguous page frames are really necessary, because contiguous linear addresses are not sufficient to satisfy the request. A typical example is a memory request for buffers to be assigned to a DMA processor (see [Chapter 13](#)). Because most DMAs ignore the paging circuitry and access the address bus directly while transferring several disk sectors in a single I/O operation, the buffers requested must be located in contiguous page frames.
- Even if contiguous page frame allocation is not strictly necessary, it offers the big advantage of leaving the kernel paging tables unchanged. What's wrong with modifying the Page Tables? As we know from [Chapter 2](#), frequent Page Table modifications lead to higher average memory access times, because they make the CPU flush the contents of the translation lookaside buffers.
- Large chunks of contiguous physical memory can be accessed by the kernel through 4 MB pages. This reduces the translation lookaside buffers misses, thus significantly speeding up the average memory access time (see the section "[Translation Lookaside Buffers \(TLB\)](#)" in [Chapter 2](#)).

The technique adopted by Linux to solve the external fragmentation problem is based on the well-known buddy system algorithm. All free page frames are grouped into 11 lists of blocks that contain groups of 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, and 1024 contiguous page frames, respectively. The largest request of 1024 page frames corresponds to a chunk of 4 MB of contiguous RAM. The physical address of the first page frame of a block is a multiple of the group size—for example, the initial address of a 16-page-frame block is a multiple of

16×2^{12} ($2^{12} = 4,096$, which is the regular page size).

We'll show how the algorithm works through a simple example:

Assume there is a request for a group of 256 contiguous page frames (i.e., one megabyte). The algorithm checks first to see whether a free block in the 256-page-frame list exists. If there is no such block, the algorithm looks for the next larger block a free block in the 512-page-frame list. If such a block exists, the kernel allocates 256 of the 512 page frames to satisfy the request and inserts the remaining 256 page frames into the list of free 256-page-frame blocks. If there is no free 512-page block, the kernel then looks for the next larger block (i.e., a free 1024-page-frame block). If such a block exists, it allocates 256 of the 1024 page frames to satisfy the request, inserts the first 512 of the remaining 768 page frames into the list of free 512-page-frame blocks, and inserts the last 256 page frames into the list of free 256-page-frame blocks. If the list of 1024-page-frame blocks is empty, the algorithm gives up and signals an error condition.

The reverse operation, releasing blocks of page frames, gives rise to the name of this algorithm. The kernel attempts to merge pairs of free buddy blocks of size b together into a single block of size $2b$. Two blocks are considered buddies if:

- Both blocks have the same size, say b .
- They are located in contiguous physical addresses.
- The physical address of the first page frame of the first block is a multiple of $2 \times b \times 2^{12}$.

The algorithm is iterative; if it succeeds in merging released blocks, it doubles b and tries again so as to create even bigger blocks.

8.1.7.1. Data structures

Linux 2.6 uses a different buddy system for each zone. Thus, in the 80 x 86 architecture, there are 3 buddy systems: the first handles the page frames suitable for ISA DMA, the second handles the "normal" page frames, and the third handles the high-memory page frames. Each buddy system relies on the following main data structures :

- The `mem_map` array introduced previously. Actually, each zone is concerned with a subset of the `mem_map` elements. The first element in the subset and its number of elements are specified, respectively, by the `zone_mem_map` and `size` fields of the zone descriptor.
- An array consisting of eleven elements of type `free_area`, one element for each group size. The array is stored in the `free_area` field of the zone descriptor.

Let us consider the k^{th} element of the `free_area` array in the zone descriptor, which identifies all the free blocks of size 2^k . The `free_list` field of this element is the head of a doubly linked circular list that collects the page descriptors associated with the free blocks of 2^k pages. More precisely, this list includes the page descriptors of the starting page frame of every block of 2^k free page frames; the pointers to the adjacent elements in the list are stored in the `lru` field of the page descriptor.^[*]

[*] As we'll see later, the `lru` field of the page descriptor can be used with other meanings when the page is not free.

Besides the head of the list, the k^{th} element of the `free_area` array includes also the field `nr_free`, which specifies the number of free blocks of size 2^k pages. Of course, if there are no blocks of 2^k free page frames, `nr_free` is equal to 0 and the `free_list` list is empty (both pointers of `free_list` point to the `free_list` field itself).

Finally, the `private` field of the descriptor of the first page in a block of 2^k free pages stores the order of the block, that is, the number k . Thanks to this field, when a block of pages is freed, the kernel can determine whether the buddy of the block is also free and, if so, it can coalesce the two blocks in a single block of 2^{k+1} pages. It should be noted that up to Linux 2.6.10, the kernel used 10 arrays of flags to encode this information.

8.1.7.2. Allocating a block

The `__rmqueue( )` function is used to find a free block in a zone. The function takes two arguments: the address of the zone descriptor, and `order`, which denotes the logarithm of the size of the requested block of free pages (0 for a one-page block, 1 for a two-page block, and so forth). If the page frames are successfully allocated, the `__rmqueue( )` function returns the address of the page descriptor of the first allocated page frame. Otherwise, the function returns `NULL`.

The `__rmqueue( )` function assumes that the caller has already disabled local interrupts and acquired the `zone->lock` spin lock, which protects the data structures of the buddy system. It performs a cyclic search through each list for an available block (denoted by an entry that doesn't point to the entry itself), starting with the list for the requested `order` and continuing if necessary to larger orders:

```
struct free_area *area;
unsigned int current_order;

for (current_order=order; current_order<11; ++current_order) {
    area = zone->free_area + current_order;
    if (!list_empty(&area->free_list))
        goto block_found;
}
return NULL;
```

If the loop terminates, no suitable free block has been found, so `__rmqueue( )` returns a `NULL` value. Otherwise, a suitable free block has been found; in this case, the descriptor of its first page frame is removed from the list and the value of `free_ pages` in the zone descriptor is decreased:

```
block_found:
    page = list_entry(area->free_list.next, struct page, lru);
    list_del(&page->lru);
    ClearPagePrivate(page);
    page->private = 0;
    area->nr_free--;
    zone->free_pages -= 1UL << order;
```

If the block found comes from a list of size `curr_order` greater than the requested size `order`, a while cycle is executed. The rationale behind these lines of codes is as follows: when it becomes necessary to use a block of 2^k page frames to satisfy a request for 2^h page frames ($h < k$), the program allocates the first 2^h page frames and iteratively reassigns the last $2^k - 2^h$ page frames to the `free_area` lists that have indexes between h and k :

```
size = 1 << curr_order;
while (curr_order > order) {
    area--;
    curr_order--;
    size >>= 1;
    buddy = page + size;
```

```

/* insert buddy as first element in the list */
list_add(&buddy->lru, &area->free_list);
area->nr_free++;
buddy->private = curr_order;
SetPagePrivate(buddy);
}
return page;

```

Because the `__rmqueue( )` function has found a suitable free block, it returns the address page of the page descriptor associated with the first allocated page frame.

8.1.7.3. Freeing a block

The `__free_pages_bulk( )` function implements the buddy system strategy for freeing page frames. It uses three basic input parameters:^[*]

[*] For performance reasons, this inline function also uses another parameter; its value, however, can be determined by the three basic parameters shown in the text.

page

The address of the descriptor of the first page frame included in the block to be released

zone

The address of the zone descriptor

order

The logarithmic size of the block

The function assumes that the caller has already disabled local interrupts and acquired the `zone->lock` spin lock, which protects the data structure of the buddy system. `__free_pages_bulk( )` starts by declaring and initializing a few local variables:

```

struct page * base = zone->zone_mem_map;
unsigned long buddy_idx, page_idx = page - base;
struct page * buddy, * coalesced;
int order_size = 1 << order;

```

The `page_idx` local variable contains the index of the first page frame in the block with respect to the first page frame of the zone.

The `order_size` local variable is used to increase the counter of free page frames in the zone:

```

zone->free_pages += order_size;

```

The function now performs a cycle executed at most `10 - order` times, once for each possibility for merging a block with its buddy. The function starts with the smallest-sized block and moves up to the top size:

```
while (order < 10) {
    buddy_idx = page_idx ^ (1 << order);
    buddy = base + buddy_idx;
    if (!page_is_buddy(buddy, order))
        break;
    list_del(&buddy->lru);
    zone->free_area[order].nr_free--;
    ClearPagePrivate(buddy);
    buddy->private = 0;
    page_idx ^= buddy_idx;
    order++;
}
```

In the body of the loop, the function looks for the index `buddy_idx` of the block, which is buddy to the one having the page descriptor index `page_idx`. It turns out that this index can be easily computed as:

```
buddy_idx = page_idx ^ (1 << order);
```

In fact, an Exclusive OR (XOR) using the `(1 << order)` mask switches the value of the `order`-th bit of `page_idx`. Therefore, if the bit was previously zero, `buddy_idx` is equal to `page_idx + order_size`; conversely, if the bit was previously one, `buddy_idx` is equal to `page_idx - order_size`.

Once the buddy block index is known, the page descriptor of the buddy block can be easily obtained as:

```
buddy = base + buddy_idx;
```

Now the function invokes `page_is_buddy()` to check if `buddy` describes the first page of a block of `order_size` free page frames.

```
int page_is_buddy(struct page *page, int order)
{
    if (PagePrivate(buddy) && page->private == order &&
        !PageReserved(buddy) && page_count(page) == 0)
        return 1;
    return 0;
}
```

As you see, the buddy's first page must be free (`_count` field equal to `-1`), it must belong to the dynamic memory (`PG_reserved` bit clear), its `private` field must be meaningful (`PG_private` bit set), and finally the `private` field must store the order of the block being freed.

If all these conditions are met, the buddy block is free and the function removes the buddy block from the list of free blocks of order `order`, and performs one more iteration looking for buddy blocks twice as big.

If at least one of the conditions in `page_is_buddy()` is not met, the function breaks out of the cycle, because the free block obtained cannot be merged further with other free blocks. The function inserts it in the proper list and updates the `private` field of the first page frame with the order of the block size:

```

coalesced = base + page_idx;
coalesced->private = order;
SetPagePrivate(coalesced);
list_add(&coalesced->lru, &zone->free_area[order].free_list);
zone->free_area[order].nr_free++;

```

8.1.8. The Per-CPU Page Frame Cache

As we will see later in this chapter, the kernel often requests and releases single page frames. To boost system performance, each memory zone defines a per-CPU page frame cache. Each per-CPU cache includes some pre-allocated page frames to be used for single memory requests issued by the local CPU.

Actually, there are two caches for each memory zone and for each CPU: a hot cache, which stores page frames whose contents are likely to be included in the CPU's hardware cache, and a cold cache.

Taking a page frame from the hot cache is beneficial for system performance if either the kernel or a User Mode process will write into the page frame right after the allocation. In fact, every access to a memory cell of the page frame will result in a line of the hardware cache being "stolen" from another page frame unless, of course, the hardware cache already includes a line that maps the cell of the "hot" page frame just accessed.

Conversely, taking a page frame from the cold cache is convenient if the page frame is going to be filled with a DMA operation. In this case, the CPU is not involved and no line of the hardware cache will be modified. Taking the page frame from the cold cache preserves the reserve of hot page frames for the other kinds of memory allocation requests.

The main data structure implementing the per-CPU page frame cache is an array of `per_cpu_pageset` data structures stored in the `pageset` field of the memory zone descriptor. The array includes one element for each CPU; this element, in turn, consists of two `per_cpu_pages` descriptors, one for the hot cache and the other for the cold cache. The fields of the `per_cpu_pages` descriptor are listed in [Table 8-7](#).

Table 8-7. The fields of the `per_cpu_pages` descriptor

Type

Name

Description

`int`

`count`

Number of pages frame in the cache

`int`

`low`

Low watermark for cache replenishing

`int`

`high`

High watermark for cache depletion

`int`

`batch`

Number of page frames to be added or subtracted from the cache

`struct list_head`

`list`

List of descriptors of the page frames included in the cache

The kernel monitors the size of the both the hot and cold caches by using two watermarks: if the number of page frames falls below the `low` watermark, the kernel replenishes the proper cache by allocating `batch` single page frames from the buddy system; otherwise, if the number of page frames rises above the `high` watermark, the kernel releases to the buddy system `batch` page frames in the cache. The values of `batch`, `low`, and `high` essentially depend on the number of page frames included in the memory zone.

8.1.8.1. Allocating page frames through the per-CPU page frame caches

The `buffered_rmqueue( )` function allocates page frames in a given memory zone. It makes use of the per-CPU page frame caches to handle single page frame requests.

The parameters are the address of the memory zone descriptor, the order of the memory allocation request `order`, and the allocation flags `gfp_flags`. If the `__GFP_COLD` flag is set in `gfp_flags`, the page frame should be taken from the cold cache, otherwise it should be taken from the hot cache (this flag is meaningful only for single page frame requests). The function essentially executes the following operations:

1. If `order` is not equal to 0, the per-CPU page frame cache cannot be used: the function jumps to step 4.
2. Checks whether the memory zone's local per-CPU cache identified by the value of the `__GFP_COLD` flag has to be replenished (the `count` field of the `per_cpu_pages` descriptor is lower than or equal to the `low` field). In this case, it executes the following substeps:
 - a. Allocates `batch` single page frames from the buddy system by repeatedly invoking the `__rmqueue( )` function.
 - b. Inserts the descriptors of the allocated page frames in the cache's list.
 - c. Updates the value of `count` by adding the number of page frames actually allocated.
3. If `count` is positive, the function gets a page frame from the cache's list, decreases `count`, and jumps to step 5. (Observe that a per-CPU page frame cache could be empty; this happens when the `__rmqueue( )` function invoked in step 2a fails to allocate any page frames.)
4. Here, the memory request has not yet been satisfied, either because the request spans several contiguous page frames, or because the selected page frame cache is empty. Invokes the `__rmqueue( )` function to allocate the requested page frames from the buddy system.

5. If the memory request has been satisfied, the function initializes the page descriptor of the (first) page frame: clears some flags, sets the `private` field to zero, and sets the page frame reference counter to one. Moreover, if the `__GPF_ZERO` flag in `gfp_flags` is set, it fills the allocated memory area with zeros.
6. Returns the page descriptor address of the (first) page frame, or `NULL` if the memory allocation request failed.

8.1.8.2. Releasing page frames to the per-CPU page frame caches

In order to release a single page frame to a per-CPU page frame cache, the kernel makes use of the `free_hot_page( )` and `free_cold_page( )` functions. Both of them are simple wrappers for the `free_hot_cold_page( )` function, which receives as its parameters the descriptor address `page` of the page frame to be released and a `cold` flag specifying either the hot cache or the cold cache.

The `free_hot_cold_page( )` function executes the following operations:

1. Gets from the `page->flags` field the address of the memory zone descriptor including the page frame (see the earlier section "[Non-Uniform Memory Access \(NUMA\)](#)").
2. Gets the address of the `per_cpu_pages` descriptor of the zone's cache selected by the `cold` flag.
3. Checks whether the cache should be depleted: if `count` is higher than or equal to `high`, invokes the `free_pages_bulk( )` function, passing to it the zone descriptor, the number of page frames to be released (`batch` field), the address of the cache's list, and the number zero (for 0-order page frames). In turn, the latter function invokes repeatedly the `__free_pages_bulk( )` function to releases the specified number of page frames taken from the cache's list to the buddy system of the memory zone.
4. Adds the page frame to be released to the cache's list, and increases the `count` field.

It should be noted that in the current version of the Linux 2.6 kernel, no page frame is ever released to the cold cache: the kernel always assumes the freed page frame is hot with respect to the hardware cache. Of course, this does not mean that the cold cache is empty: the cache is replenished by `buffered_rmqueue( )` when the low watermark has been reached.

8.1.9. The Zone Allocator

The zone allocator is the frontend of the kernel page frame allocator. This component must locate a memory zone that includes a number of free page frames large enough to satisfy the memory request. This task is not as simple as it could appear at a first glance, because the zone allocator must satisfy several goals:

- It should protect the pool of reserved page frames (see the earlier section "[The Pool of Reserved Page Frames](#)").
- It should trigger the page frame reclaiming algorithm (see [Chapter 17](#)) when memory is scarce and blocking the current process is allowed; once some page frames have been freed, the zone allocator will retry the allocation.
- It should preserve the small, precious `ZONE_DMA` memory zone, if possible. For instance, the zone allocator should be somewhat reluctant to assign page frames in the `ZONE_DMA` memory zone if the request was for `ZONE_NORMAL` or `ZONE_HIGHMEM` page frames.

We have seen in the earlier section "[The Zoned Page Frame Allocator](#)" that every request for a group of contiguous page frames is eventually handled by executing the `alloc_pages` macro. This macro, in turn, ends up invoking the `__alloc_pages( )` function, which is the core of the zone allocator. It receives three parameters:

`gfp_mask`

The flags specified in the memory allocation request (see earlier [Table 8-5](#))

`order`

The logarithmic size of the group of contiguous page frames to be allocated

`zonelist`

Pointer to a `zonelist` data structure describing, in order of preference, the memory zones suitable for the memory allocation

The `__alloc_pages( )` function scans every memory zone included in the `zonelist` data structure. The code that does this looks like the following:

```
for (i = 0; (z=zonelist->zones[i]) != NULL; i++) {
    if (zone_watermark_ok(z, order, ...)) {
        page = buffered_rmqueue(z, order, gfp_mask);
        if (page)
            return page;
    }
}
```

For each memory zone, the function compares the number of free page frames with a threshold value that depends on the memory allocation flags, on the type of current process, and on how many times the zone has already been checked by the function. In fact, if free memory is scarce, every memory zone is typically scanned several times, each time with lower threshold on the minimal amount of free memory required for the allocation. The previous block of code is thus replicated several times with minor variations in the body of the `__alloc_pages( )` function. The `buffered_rmqueue( )` function has been described already in the earlier section "[The Per-CPU Page Frame Cache](#):" it returns the page descriptor of the first allocated page frame, or `NULL` if the memory zone does not include a group of contiguous page frames of the requested size.

The `zone_watermark_ok( )` auxiliary function receives several parameters, which determine a threshold `min` on the number of free page frames in the memory zone. In particular, the function returns the value 1 if the following two conditions are met:

1. Besides the page frames to be allocated, there are at least `min` free page frames in the memory zone, not including the page frames in the low-on-memory reserve (`lowmem_reserve` field of the zone descriptor).
2. Besides the page frames to be allocated, there are at least $\frac{\text{min}}{2^k}$ free page frames in blocks of order at least `k`, for each `k` between 1 and the order of the allocation. Therefore, if `order` is greater than zero, there must be at least `min/2` free page frames in blocks of size at least 2; if `order` is greater than one, there must be at least `min/4` free page frames in blocks of size at least 4; and so on.

The value of the threshold `min` is determined by `zone_watermark_ok( )` as follows:

- The base value is passed as a parameter of the function and can be one of the `pages_min`, `pages_low`, and `pages_high` zone's watermarks (see the section "[The Pool of Reserved Page Frames](#)" earlier in this chapter).
- The base value is divided by two if the `gfp_high` flag passed as parameter is set. Usually, this flag is equal to one if the `__GFP_HIGHMEM` flag is set in the `gfp_mask`, that is, if the page frames can be allocated from high memory.

- The threshold value is further reduced by one-fourth if the `can_try_harder` flag passed as parameter is set. This flag is usually equal to one if either the `__GFP_WAIT` flag is set in `gfp_mask`, or if the current process is a real-time process and the memory allocation is done in process context (outside of interrupt handlers and deferrable functions).

The `__alloc_pages()` function essentially executes the following steps:

1. Performs a first scanning of the memory zones (see the block of code shown earlier). In this first scan, the `min` threshold value is set to `z->pages_low`, where `z` points to the zone descriptor being analyzed (the `can_try_harder` and `gfp_high` parameters are set to zero).
2. If the function did not terminate in the previous step, there is not much free memory left: the function awakens the `kswapd` kernel threads to start reclaiming page frames asynchronously (see [Chapter 17](#)).
3. Performs a second scanning of the memory zones, passing as base threshold the value `z->pages_min`. As explained previously, the actual threshold is determined also by the `can_try_harder` and `gfp_high` flags. This step is nearly identical to step 1, except that the function is using a lower threshold.
4. If the function did not terminate in the previous step, the system is definitely low on memory. If the kernel control path that issued the memory allocation request is not an interrupt handler or a deferrable function and it is trying to reclaim page frames (either the `PF_MEMALLOC` flag or the `PF_MEMDIE` flag of `current` is set), the function then performs a third scanning of the memory zones, trying to allocate the page frames ignoring the low-on-memory threshold that is, without invoking `zone_watermark_ok()`. This is the only case where the kernel control path is allowed to deplete the low-on-memory reserve of pages specified by the `lowmem_reserve` field of the zone descriptor. In fact, in this case the kernel control path that issued the memory request is ultimately trying to free page frames, thus it should get what it has requested, if at all possible. If no memory zone includes enough page frames, the function returns `NULL` to notify the caller of the failure.
5. Here, the invoking kernel control path is not trying to reclaim memory. If the `__GFP_WAIT` flag of `gfp_mask` is not set, the function returns `NULL` to notify the kernel control path of the memory allocation failure: in this case, there is no way to satisfy the request without blocking the current process.
6. Here the current process can be blocked: invokes `cond_resched()` to check whether some other process needs the CPU.
7. Sets the `PF_MEMALLOC` flag of `current`, to denote the fact that the process is ready to perform memory reclaiming.
8. Stores in `current->reclaim_state` a pointer to a `reclaim_state` structure. This structure includes just one field, `reclaimed_slab`, initialized to zero (we'll see how this field is used in the section ["Interfacing the Slab Allocator with the Zoned Page Frame Allocator"](#) later in this chapter).
9. Invokes `Try_to_free_pages()` to look for some page frames to be reclaimed (see the section ["Low On Memory Reclaiming"](#) in [Chapter 17](#)). The latter function may block the current process. Once that function returns, `__alloc_pages()` resets the `PF_MEMALLOC` flag of `current` and invokes once more `cond_resched()`.
10. If the previous step has freed some page frames, the function performs yet another scanning of the memory zones equal to the one performed in step 3. If the memory allocation request cannot be satisfied, the function determines whether it should continue scanning the memory zone: if the `__GFP_NORETRY` flag is clear and either the memory allocation request spans up to eight page frames, or one of the `__GFP_REPEAT` and `__GFP_NOFAIL` flags is set, the function invokes `blk_congestion_wait()` to put the process asleep for awhile (see [Chapter 14](#)), and it jumps back to step 6. Otherwise, the function returns `NULL` to notify the caller that the memory allocation failed.
11. If no page frame has been freed in step 9, the kernel is in deep trouble, because free memory is dangerously low and it was not possible to reclaim any page frame. Perhaps the time has come to take a crucial decision. If the kernel control path is allowed to perform the filesystem-dependent operations needed to kill a process (the `__GFP_FS` flag in `gfp_mask` is set) and the `__GFP_NORETRY` flag is clear, performs the following substeps:

- a. Scans once again the memory zones with a threshold value equal to `z->pages_high`.
- b. Invokes `out_of_memory()` to start freeing some memory by killing a victim process (see ["The Out of Memory Killer"](#) in [Chapter 17](#)).
- c. Jumps back to step 1.

Because the watermark used in step 11a is much higher than the watermarks used in the previous scannings, that step is likely to fail. Actually, step 11a succeeds only if another kernel control path is already killing a process to reclaim its memory. Thus, step 11a avoids that two innocent processes are killed instead of one.

8.1.9.1. Releasing a group of page frames

The zone allocator also takes care of releasing page frames; thankfully, releasing memory is a lot easier than allocating it.

All kernel macros and functions that release page frames described in the earlier section ["The Zoned Page Frame Allocator"](#) rely on the `__free_pages()` function. It receives as its parameters the address of the page descriptor of the first page frame to be released (`page`), and the logarithmic size of the group of contiguous page frames to be released (`order`). The function executes the following steps:

1. Checks that the first page frame really belongs to dynamic memory (its `PG_reserved` flag is cleared); if not, terminates.
2. Decreases the `page->_count` usage counter; if it is still greater than or equal to zero, terminates.
3. If `order` is equal to zero, the function invokes `free_hot_page()` to release the page frame to the per-CPU hot cache of the proper memory zone (see the earlier section ["The Per-CPU Page Frame Cache"](#)).
4. If `order` is greater than zero, it adds the page frames in a local list and invokes the `free_pages_bulk()` function to release them to the buddy system of the proper memory zone (see step 3 in the description of `free_hot_cold_page()` in the earlier section ["The Per-CPU Page Frame Cache"](#)).

8.2. Memory Area Management

This section deals with memory areas that is, with sequences of memory cells having contiguous physical addresses and an arbitrary length.

The buddy system algorithm adopts the page frame as the basic memory area. This is fine for dealing with relatively large memory requests, but how are we going to deal with requests for small memory areas, say a few tens or hundreds of bytes?

Clearly, it would be quite wasteful to allocate a full page frame to store a few bytes. A better approach instead consists of introducing new data structures that describe how small memory areas are allocated within the same page frame. In doing so, we introduce a new problem called internal fragmentation. It is caused by a mismatch between the size of the memory request and the size of the memory area allocated to satisfy the request.

A classical solution (adopted by early Linux versions) consists of providing memory areas whose sizes are geometrically distributed; in other words, the size depends on a power of 2 rather than on the size of the data to be stored. In this way, no matter what the memory request size is, we can ensure that the internal fragmentation is always smaller than 50 percent. Following this approach, the kernel creates 13 geometrically distributed lists of free memory areas whose sizes range from 32 to 131, 072 bytes. The buddy system is invoked both to obtain additional page frames needed to store new memory areas and, conversely, to release page frames that no longer contain memory areas. A dynamic list is used to keep track of the free memory areas contained in each page frame.

8.2.1. The Slab Allocator

Running a memory area allocation algorithm on top of the buddy algorithm is not particularly efficient. A better algorithm is derived from the slab allocator schema that was adopted for the first time in the Sun Microsystems Solaris 2.4 operating system. It is based on the following premises:

- The type of data to be stored may affect how memory areas are allocated; for instance, when allocating a page frame to a User Mode process, the kernel invokes the `get_zeroed_page( )` function, which fills the page with zeros.

The concept of a slab allocator expands upon this idea and views the memory areas as objects consisting of both a set of data structures and a couple of functions or methods called the constructor and destructor. The former initializes the memory area while the latter deinitializes it.

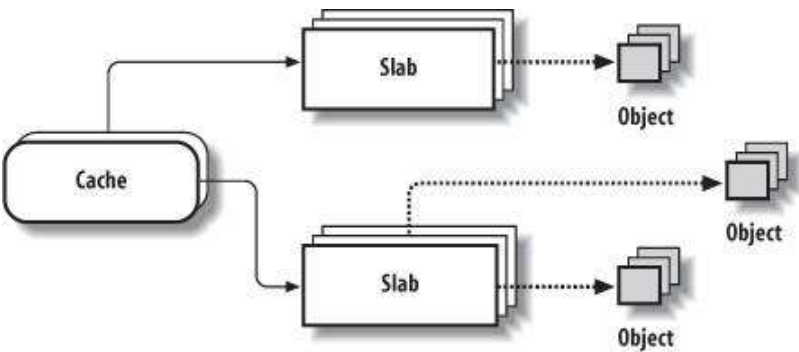
To avoid initializing objects repeatedly, the slab allocator does not discard the objects that have been allocated and then released but instead saves them in memory. When a new object is then requested, it can be taken from memory without having to be reinitialized.

- The kernel functions tend to request memory areas of the same type repeatedly. For instance, whenever the kernel creates a new process, it allocates memory areas for some fixed size tables such as the process descriptor, the open file object, and so on (see [Chapter 3](#)). When a process terminates, the memory areas used to contain these tables can be reused. Because processes are created and destroyed quite frequently, without the slab allocator, the kernel wastes time allocating and deallocating the page frames containing the same memory areas repeatedly; the slab allocator allows them to be saved in a cache and reused quickly.
- Requests for memory areas can be classified according to their frequency. Requests of a particular size that are expected to occur frequently can be handled most efficiently by creating a set of special-purpose objects that have the right size, thus avoiding internal fragmentation. Meanwhile, sizes that are rarely encountered can be handled through an allocation scheme based on objects in a series of geometrically distributed sizes (such as the power-of-2 sizes used in early Linux versions), even if this approach leads to internal fragmentation.
- There is another subtle bonus in introducing objects whose sizes are not geometrically distributed: the initial addresses of the data structures are less prone to be concentrated on physical addresses whose values are a power of 2. This, in turn, leads to better performance by the processor hardware cache.
- Hardware cache performance creates an additional reason for limiting calls to the buddy system allocator as much as possible. Every call to a buddy system function "dirties" the hardware cache, thus increasing the average memory access time. The impact of a kernel function on the hardware cache is called the function footprint; it is defined as the percentage of cache overwritten by the function when it terminates. Clearly, large footprints lead to a slower execution of the code executed right after the kernel function, because the hardware cache is by now filled with useless information.

The slab allocator groups objects into caches. Each cache is a "store" of objects of the same type. For instance, when a file is opened, the memory area needed to store the corresponding "open file" object is taken from a slab allocator cache named filp (for "file pointer").

The area of main memory that contains a cache is divided into slabs; each slab consists of one or more contiguous page frames that contain both allocated and free objects (see [Figure 8-3](#)).

Figure 8-3. The slab allocator components



As we'll see in [Chapter 17](#), the kernel periodically scans the caches and releases the page frames corresponding to empty slabs.

8.2.2. Cache Descriptor

Each cache is described by a structure of type `kmem_cache_t` (which is equivalent to the type `struct kmem_cache_s`), whose fields are listed in [Table 8-8](#). We omitted from the table several fields used for collecting statistical information and for debugging.

Table 8-8. The fields of the `kmem_cache_t` descriptor

Type	Name	Description
<code>struct</code> <code>array_cache * []</code>	array	Per-CPU array of pointers to local caches of free objects (see the section " Local Caches of Free Slab Objects " later in this chapter).
<code>unsigned int</code>	batchcount	Number of objects to be transferred in bulk to or from the local caches.
<code>unsigned int</code>	limit	Maximum number of free objects in the local caches. This is tunable.

<code>struct kmem_list3</code>	<code>lists</code>	See next table.
<code>unsigned int</code>	<code>objsize</code>	Size of the objects included in the cache.
<code>unsigned int</code>	<code>flags</code>	Set of flags that describes permanent properties of the cache.
<code>unsigned int</code>	<code>num</code>	Number of objects packed into a single slab. (All slabs of the cache have the same size.)
<code>unsigned int</code>	<code>free_limit</code>	Upper limit of free objects in the whole slab cache.
<code>spinlock_t</code>	<code>spinlock</code>	Cache spin lock.
<code>unsigned int</code>	<code>gfporder</code>	Logarithm of the number of contiguous page frames included in a single slab.
<code>unsigned int</code>	<code>gfpflags</code>	Set of flags passed to the buddy system function when allocating page frames.
<code>size_t</code>	<code>colour</code>	Number of colors for the slabs (see the section " Slab Coloring " later in this chapter).
<code>unsigned int</code>	<code>colour_off</code>	Basic alignment offset in the slabs.
<code>unsigned int</code>	<code>colour_next</code>	Color to use for the next allocated slab.

		Pointer to the general slab cache containing the slab descriptors (NULL if internal slab descriptors are used; see next section).
<code>kmem_cache_t *</code>	<code>slabp_cache</code>	
<code>unsigned int</code>	<code>slab_size</code>	The size of a single slab.
<code>unsigned int</code>	<code>dflags</code>	Set of flags that describe dynamic properties of the cache.
<code>void *</code>	<code>ctor</code>	Pointer to constructor method associated with the cache.
<code>void *</code>	<code>dtor</code>	Pointer to destructor method associated with the cache.
<code>const char *</code>	<code>name</code>	Character array storing the name of the cache.
<code>struct list_head</code>	<code>next</code>	Pointers for the doubly linked list of cache descriptors.

The `lists` field of the `kmem_cache_t` descriptor, in turn, is a structure whose fields are listed in [Table 8-9](#).

Table 8-9. The fields of the `kmem_list3` structure

Type

Name

Description

`struct list_head`

`slabs_partial`

Doubly linked circular list of slab descriptors with both free and nonfree objects

`struct list_head`

`slabs_full`

Doubly linked circular list of slab descriptors with no free objects

`struct list_head`

`slabs_free`

Doubly linked circular list of slab descriptors with free objects only

`unsigned long`

`free_objects`

Number of free objects in the cache

`int`

`free_touched`

Used by the slab allocator's page reclaiming algorithm (see [Chapter 17](#))

`unsigned long`

`next_reap`

Used by the slab allocator's page reclaiming algorithm (see [Chapter 17](#))

`struct`

`array_cache *`

`shared`

Pointer to a local cache shared by all CPUs (see the later section "[Local Caches of Free Slab Objects](#)")

8.2.3. Slab Descriptor

Each slab of a cache has its own descriptor of type `slab` illustrated in [Table 8-10](#).

Table 8-10. The fields of the slab descriptor

Type

Name

Description

`struct list_head``list`

Pointers for one of the three doubly linked list of slab descriptors (either the `slabs_full`, `slabs_partial`, or `slabs_free` list in the `kmem_list3` structure of the cache descriptor)

`unsigned long``colouroff`

Offset of the first object in the slab (see the section "[Slab Coloring](#)" later in this chapter)

`void *``s_mem`

Address of first object (either allocated or free) in the slab

`unsigned int``inuse`

Number of objects in the slab that are currently used (not free)

`unsigned int``free`

Index of next free object in the slab, or `BUFCTL_END` if there are no free objects left (see the section "[Object Descriptor](#)" later in this chapter)

Slab descriptors can be stored in two possible places:

External slab descriptor

Stored outside the slab, in one of the general caches not suitable for ISA DMA pointed to by `cache_sizes` (see the next section).

Internal slab descriptor

Stored inside the slab, at the beginning of the first page frame assigned to the slab.

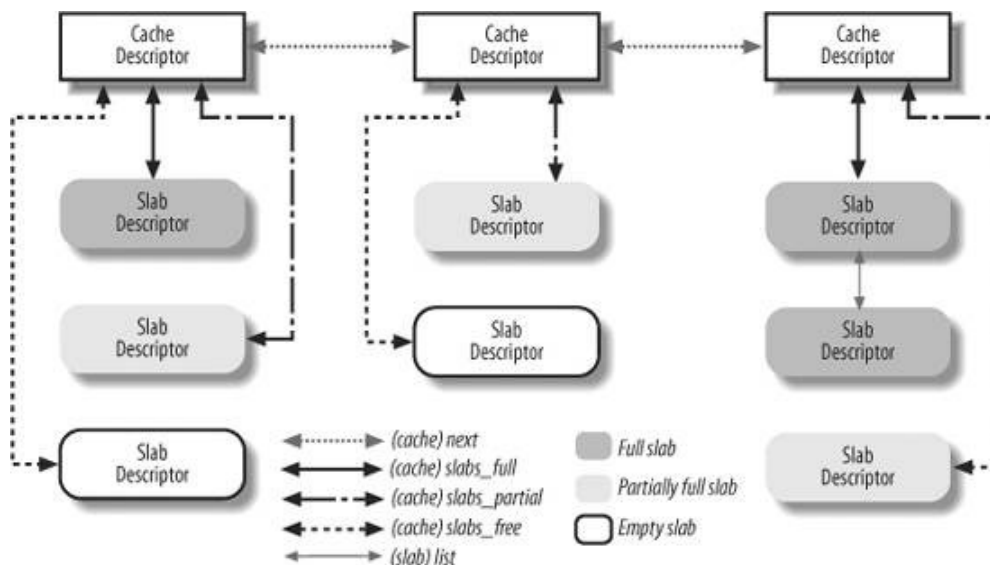
The slab allocator chooses the second solution when the size of the objects is smaller than 512MB or when internal fragmentation leaves enough space for the slab descriptor and the object descriptors (as described later) inside the slab. The `CFLGS_OFF_SLAB` flag in the `flags` field of the cache descriptor is set to one if the slab descriptor is stored outside the slab; it is set to zero otherwise.

Figure 8-4 illustrates the major relationships between cache and slab descriptors. Full slabs, partially full slabs, and free slabs are linked in different lists.

8.2.4. General and Specific Caches

Caches are divided into two types: general and specific. General caches are used only by the slab allocator for its own purposes, while specific caches are used by the remaining parts of the kernel.

Figure 8-4. Relationship between cache and slab descriptors



The general caches are:

- A first cache called `kmem_cache` whose objects are the cache descriptors of the remaining caches used by the kernel. The `cache_cache` variable contains the descriptor of this special cache.
- Several additional caches contain general purpose memory areas. The range of the memory area sizes typically includes 13 geometrically distributed sizes. A table called `malloc_sizes` (whose elements are of type `cache_sizes`) points to 26 cache descriptors associated with memory areas of size 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384, 32,768, 65,536, and 131,072 bytes. For each size, there are two caches: one suitable for ISA DMA allocations and the other for normal allocations.

The `kmem_cache_init()` function is invoked during system initialization to set up the general caches.

Specific caches are created by the `kmem_cache_create()` function. Depending on the parameters, the function first determines the best way to handle the new cache (for instance, whether to include the slab descriptor inside or outside of the slab). It then allocates a cache descriptor for the new cache from the

`cache_cache` general cache and inserts the descriptor in the `cache_chain` list of cache descriptors (the insertion is done after having acquired the `cache_chain_sem` semaphore that protects the list from concurrent accesses).

It is also possible to destroy a cache and remove it from the `cache_chain` list by invoking `kmem_cache_destroy( )`. This function is mostly useful to modules that create their own caches when loaded and destroy them when unloaded. To avoid wasting memory space, the kernel must destroy all slabs before destroying the cache itself. The `kmem_cache_shrink( )` function destroys all the slabs in a cache by invoking `slab_destroy( )` iteratively (see the later section "[Releasing a Slab from a Cache](#)").

The names of all general and specific caches can be obtained at runtime by reading `/proc/slabinfo`; this file also specifies the number of free objects and the number of allocated objects in each cache.

8.2.5. Interfacing the Slab Allocator with the Zoned Page Frame Allocator

When the slab allocator creates a new slab, it relies on the zoned page frame allocator to obtain a group of free contiguous page frames. For this purpose, it invokes the `kmem_getpages( )` function, which is essentially equivalent, on a UMA system, to the following code fragment:

```
void * kmem_getpages(kmem_cache_t *cachep, int flags)
{
    struct page *page;
    int i;

    flags |= cachep->gfpflags;
    page = alloc_pages(flags, cachep->gfporder);
    if (!page)
        return NULL;
    i = (1 << cachep->gfporder);
    if (cachep->flags & SLAB_RECLAIM_ACCOUNT)
        atomic_add(i, &slab_reclaim_pages);
    while (i--)
        SetPageSlab(page++);
    return page_address(page);
}
```

The two parameters have the following meaning:

cachep

Points to the cache descriptor of the cache that needs additional page frames (the number of required page frames is determined by the order in the `cachep->gfporder` field).

flags

Specifies how the page frame is requested (see the section "[The Zoned Page Frame Allocator](#)" earlier in this chapter). This set of flags is combined with the specific cache allocation flags stored in the `gfpflags` field of the cache descriptor.

The size of the memory allocation request is specified by the `gfporder` field of the cache descriptor, which encodes the size of a slab in the cache.^[*] If the slab cache has been created with the

SLAB_RECLAIM_ACCOUNT flag set, the page frames assigned to the slabs are accounted for as reclaimable pages when the kernel checks whether there is enough memory to satisfy some User Mode requests. The function also sets the PG_slab flag in the page descriptors of the allocated page frames.

[*] Notice that it is not possible to allocate page frames from the ZONE_HIGHMEM memory zone, because the `kmem_getpages( )` function returns the linear address yielded by the `page_address( )` function; as explained in the section "[Kernel Mappings of High-Memory Page Frames](#)" earlier in this chapter, this function returns NULL for unmapped high-memory page frames.

In the reverse operation, page frames assigned to a slab can be released (see the section "[Releasing a Slab from a Cache](#)" later in this chapter) by invoking the `kmem_freepages( )` function:

```
void kmem_freepages(kmem_cache_t *cachep, void *addr)
{
    unsigned long i = (1<<cachep->gfporder);
    struct page *page = virt_to_page(addr);

    if (current->reclaim_state)
        current->reclaim_state->reclaimed_slab += i;
    while (i--)
        ClearPageSlab(page++);
    free_pages((unsigned long) addr, cachep->gfporder);
    if (cachep->flags & SLAB_RECLAIM_ACCOUNT)
        atomic_sub(1<<cachep->gfporder, &slab_reclaim_pages);
}
```

The function releases the page frames, starting from the one having the linear address `addr`, that had been allocated to the slab of the cache identified by `cachep`. If the current process is performing memory reclaiming (`current->reclaim_state` field not NULL), the `reclaimed_slab` field of the `reclaim_state` structure is properly increased, so that the pages just freed can be accounted for by the page frame reclaiming algorithm (see the section "[Low On Memory Reclaiming](#)" in [Chapter 17](#)). Moreover, if the SLAB_RECLAIM_ACCOUNT flag is set (see above), the `slab_reclaim_pages` variable is properly decreased.

8.2.6. Allocating a Slab to a Cache

A newly created cache does not contain a slab and therefore does not contain any free objects. New slabs are assigned to a cache only when both of the following are true:

- A request has been issued to allocate a new object.
- The cache does not include a free object.

Under these circumstances, the slab allocator assigns a new slab to the cache by invoking `cache_grow( )`. This function calls `kmem_getpages( )` to obtain from the zoned page frame allocator the group of page frames needed to store a single slab; it then calls `alloc_slabmgmt( )` to get a new slab descriptor. If the `CFLGS_OFF_SLAB` flag of the cache descriptor is set, the slab descriptor is allocated from the general cache pointed to by the `slabp_cache` field of the cache descriptor; otherwise, the slab descriptor is allocated in the first page frame of the slab.

The kernel must be able to determine, given a page frame, whether it is used by the slab allocator and, if so, to derive quickly the addresses of the corresponding cache and slab descriptors. Therefore, `cache_grow( )`

scans all page descriptors of the page frames assigned to the new slab, and loads the `next` and `prev` subfields of the `lru` fields in the page descriptors with the addresses of, respectively, the cache descriptor and the slab descriptor. This works correctly because the `lru` field is used by functions of the buddy system only when the page frame is free, while page frames handled by the slab allocator functions have the `PG_slab` flag set and are not free as far as the buddy system is concerned.^[*] The opposite question given a slab in a cache, which are the page frames that implement it? can be answered by using the `s_mem` field of the slab descriptor and the `gfporder` field (the size of a slab) of the cache descriptor.

[*] As we'll in [Chapter 17](#), the `lru` field is also used by the page frame reclaiming algorithm.

Next, `cache_grow( )` calls `cache_init_objs( )`, which applies the constructor method (if defined) to all the objects contained in the new slab.

Finally, `cache_grow( )` calls `list_add_tail( )` to add the newly obtained slab descriptor `*slabp` at the end of the fully free slab list of the cache descriptor `*cachep`, and updates the counter of free objects in the cache:

```
list_add_tail(&slabp->list, &cachep->lists->slabs_free);
cachep->lists->free_objects += cachep->num;
```

8.2.7. Releasing a Slab from a Cache

Slabs can be destroyed in two cases:

- There are too many free objects in the slab cache (see the later section "[Releasing a Slab from a Cache](#)").
- A timer function invoked periodically determines that there are fully unused slabs that can be released (see [Chapter 17](#)).

In both cases, the `slab_destroy( )` function is invoked to destroy a slab and release the corresponding page frames to the zoned page frame allocator:

```
void slab_destroy(kmem_cache_t *cachep, slab_t *slabp)
{
    if (cachep->dtor) {
        int i;
        for (i = 0; i < cachep->num; i++) {
            void* objp = slabp->s_mem + cachep->objsize*i;
            (cachep->dtor)(objp, cachep, 0);
        }
    }
    kmem_freepages(cachep, slabp->s_mem - slabp->colouroff);
    if (cachep->flags & CFLGS_OFF_SLAB)
        kmem_cache_free(cachep->slabp_cache, slabp);
}
```

The function checks whether the cache has a destructor method for its objects (the `dtor` field is not `NULL`), in which case it applies the destructor to all the objects in the slab; the `objp` local variable keeps track of the currently examined object. Next, it calls `kmem_freepages( )`, which returns all the contiguous page frames used by the slab to the buddy system. Finally, if the slab descriptor is stored outside of the slab, the

function releases it from the cache of slab descriptors .

Actually, the function is slightly more complicated. For example, a slab cache can be created with the `SLAB_DESTROY_BY_RCU` flag, which means that slabs should be released in a deferred way by registering a callback with the `call_rcu( )` function (see the section "[Read-Copy Update \(RCU\)](#)" in [Chapter 5](#)). The callback function, in turn, invokes `kmem_freepages( )` and, possibly, the `kmem_cache_free( )`, as in the main case shown above.

8.2.8. Object Descriptor

Each object has a short descriptor of type `kmem_bufctl_t`. Object descriptors are stored in an array placed right after the corresponding slab descriptor. Thus, like the slab descriptors themselves, the object descriptors of a slab can be stored in two possible ways that are illustrated in [Figure 8-5](#).

External object descriptors

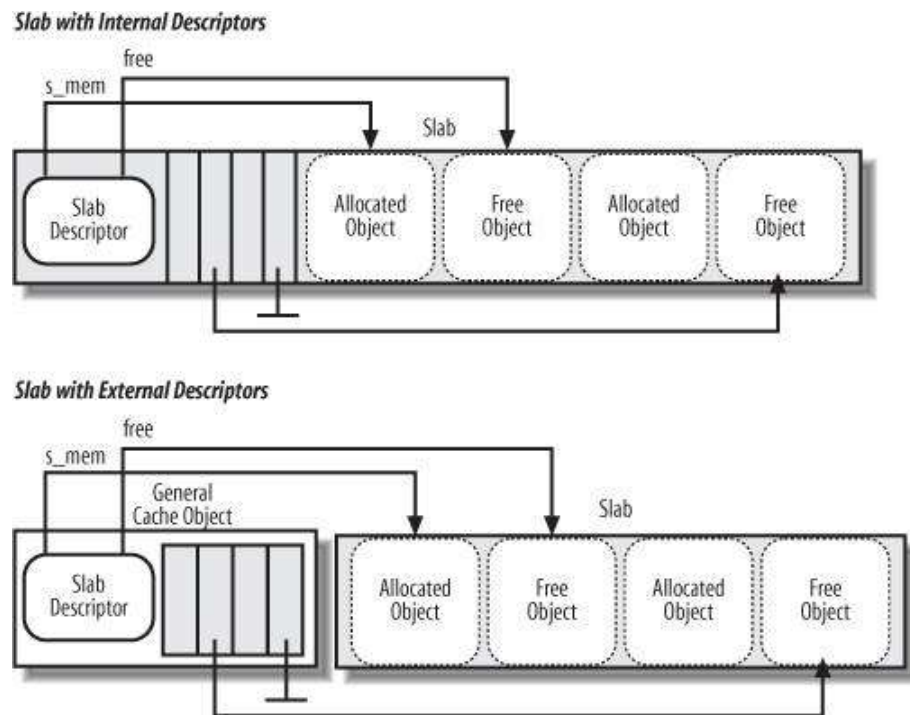
Stored outside the slab, in the general cache pointed to by the `slabp_cache` field of the cache descriptor. The size of the memory area, and thus the particular general cache used to store object descriptors, depends on the number of objects stored in the slab (`num` field of the cache descriptor).

Internal object descriptors

Stored inside the slab, right before the objects they describe.

The first object descriptor in the array describes the first object in the slab, and so on. An object descriptor is simply an unsigned short integer, which is meaningful only when the object is free. It contains the index of the next free object in the slab, thus implementing a simple list of free objects inside the slab. The object descriptor of the last element in the free object list is marked by the conventional value `BUFCTL_END` (`0xffff`).

Figure 8-5. Relationships between slab and object descriptors



8.2.9. Aligning Objects in Memory

The objects managed by the slab allocator are aligned in memory—that is, they are stored in memory cells whose initial physical addresses are multiples of a given constant, which is usually a power of 2. This constant is called the alignment factor.

The largest alignment factor allowed by the slab allocator is 4,096—the page frame size. This means that objects can be aligned by referring to either their physical addresses or their linear addresses. In both cases, only the 12 least significant bits of the address may be altered by the alignment.

Usually, microcomputers access memory cells more quickly if their physical addresses are aligned with respect to the word size (that is, to the width of the internal memory bus of the computer). Thus, by default, the `kmem_cache_create()` function aligns objects according to the word size specified by the `BYTES_PER_WORD` macro. For 80 x 86 processors, the macro yields the value 4 because the word is 32 bits long.

When creating a new slab cache, it's possible to specify that the objects included in it be aligned in the first-level hardware cache. To achieve this, the kernel sets the `SLAB_HWCACHE_ALIGN` cache descriptor flag. The `kmem_cache_create()` function handles the request as follows:

- If the object's size is greater than half of a cache line, it is aligned in RAM to a multiple of `L1_CACHE_BYTES`—that is, at the beginning of the line.
- Otherwise, the object size is rounded up to a submultiple of `L1_CACHE_BYTES`; this ensures that a small object will never span across two cache lines.

Clearly, what the slab allocator is doing here is trading memory space for access time; it gets better cache performance by artificially increasing the object size, thus causing additional internal fragmentation.

8.2.10. Slab Coloring

We know from [Chapter 2](#) that the same hardware cache line maps many different blocks of RAM. In this chapter, we have also seen that objects of the same size end up being stored at the same offset within a cache. Objects that have the same offset within different slabs will, with a relatively high probability, end up mapped in the same cache line. The cache hardware might therefore waste memory cycles transferring two objects from the same cache line back and forth to different RAM locations, while other cache lines go underutilized. The slab allocator tries to reduce this unpleasant cache behavior by a policy called slab coloring : different arbitrary values called colors are assigned to the slabs.

Before examining slab coloring, we have to look at the layout of objects in the cache. Let's consider a cache whose objects are aligned in RAM. This means that the object address must be a multiple of a given positive value, say aln . Even taking the alignment constraint into account, there are many possible ways to place objects inside the slab. The choices depend on decisions made for the following variables:

`num`

Number of objects that can be stored in a slab (its value is in the `num` field of the cache descriptor).

`osize`

Object size, including the alignment bytes.

`dsize`

Slab descriptor size plus all object descriptors size, rounded up to the smallest multiple of the hardware cache line size. Its value is equal to 0 if the slab and object descriptors are stored outside of the slab.

`free`

Number of unused bytes (bytes not assigned to any object) inside the slab.

The total length in bytes of a slab can then be expressed as:

```
slab length = (num x osize) + dsize+ free
```

`free` is always smaller than `osize`, because otherwise, it would be possible to place additional objects inside the slab. However, `free` could be greater than aln .

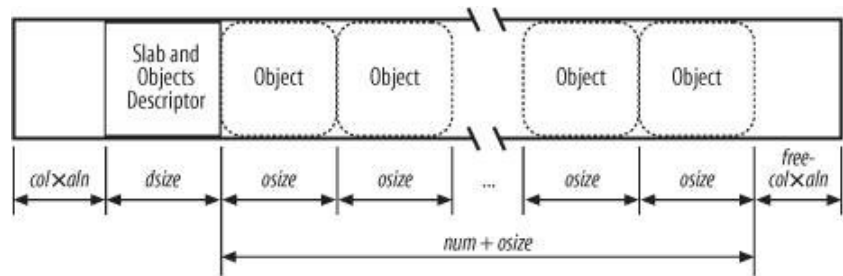
The slab allocator takes advantage of the `free` unused bytes to color the slab. The term "color" is used simply to subdivide the slabs and allow the memory allocator to spread objects out among different linear addresses. In this way, the kernel obtains the best possible performance from the microprocessor's hardware cache.

Slabs having different colors store the first object of the slab in different memory locations, while satisfying the alignment constraint. The number of available colors is $free/aln$ (this value is stored in the `colour` field of the cache descriptor). Thus, the first color is denoted as 0 and the last one is denoted as $(free / aln)-1$. (As a particular case, if `free` is lower than aln , `colour` is set to 0, nevertheless all slabs use color 0, thus really the number of colors is one.)

If a slab is colored with color `col`, the offset of the first object (with respect to the slab initial address) is equal to $col \times aln + dsize$ bytes. [Figure 8-6](#) illustrates how the placement of objects inside the slab depends on the

slab color. Coloring essentially leads to moving some of the free area of the slab from the end to the beginning.

Figure 8-6. Slab with color col and alignment aln



Coloring works only when free is large enough. Clearly, if no alignment is required for the objects or if the number of unused bytes inside the slab is smaller than the required alignment ($free < aln$), the only possible slab coloring is the one that has the color 0the one that assigns a zero offset to the first object.

The various colors are distributed equally among slabs of a given object type by storing the current color in a field of the cache descriptor called `colour_next`. The `cache_grow( )` function assigns the color specified by `colour_next` to a new slab and then increases the value of this field. After reaching `colour`, it wraps around again to 0. In this way, each slab is created with a different color from the previous one, up to the maximum available colors. The `cache_grow( )` function, moreover, gets the value `aln` from the `colour_off` field of the cache descriptor, computes $dsize$ according to the number of objects inside the slab, and finally stores the value $col \times aln + dsize$ in the `colouroff` field of the slab descriptor.

8.2.11. Local Caches of Free Slab Objects

The Linux 2.6 implementation of the slab allocator for multiprocessor systems differs from that of the original Solaris 2.4. To reduce spin lock contention among processors and to make better use of the hardware caches, each cache of the slab allocator includes a per-CPU data structure consisting of a small array of pointers to freed objects called the slab local cache . Most allocations and releases of slab objects affect the local cache only; the slab data structures get involved only when the local cache underflows or overflows. This technique is quite similar to the one illustrated in the section "The Per-CPU Page Frame Cache" earlier in this chapter.

The `array` field of the cache descriptor is an array of pointers to `array_cache` data structures, one element for each CPU in the system. Each `array_cache` data structure is a descriptor of the local cache of free objects, whose fields are illustrated in Table 8-11.

Table 8-11. The fields of the `array_cache` structure

Type
Name
Description
unsigned int

`avail`

Number of pointers to available objects in the local cache. The field also acts as the index of the first free slot in the cache.

`unsigned int`

`limit`

Size of the local cache that is, the maximum number of pointers in the local cache.

`unsigned int`

`batchcount`

Chunk size for local cache refill or emptying.

`unsigned int`

`touched`

Flag set to 1 if the local cache has been recently used.

Notice that the local cache descriptor does not include the address of the local cache itself; in fact, the local cache is placed right after the descriptor. Of course, the local cache stores the pointers to the freed objects, not the object themselves, which are always placed inside the slabs of the cache.

When creating a new slab cache, the `kmem_cache_create( )` function determines the size of the local caches (storing this value in the `limit` field of the cache descriptor), allocates them, and stores their pointers into the `array` field of the cache descriptor.

When creating a new slab cache, the `kmem_cache_create( )` function determines the size of the local caches (storing this value in the `limit` field of the cache descriptor), allocates them, and stores their pointers into the `array` field of the cache descriptor. The size depends on the size of the objects stored in the slab cache, and ranges from 1 for very large objects to 120 for small ones. Moreover, the initial value of the `batchcount` field, which is the number of objects added or removed in a chunk from a local cache, is initially set to half of the local cache size.^[*]

[*] The system administrator can tune for each cache the size of the local caches and the value of the `batchcount` field by writing into the `/proc/slabinfo` file.

In multiprocessor systems, slab caches for small objects also sport an additional local cache, whose address is stored in the `lists.shared` field of the cache descriptor. The shared local cache is, as the name suggests, shared among all CPUs, and it makes the task of migrating free objects from a local cache to another easier (see the following section). Its initial size is equal to eight times the value of the `batchcount` field.

8.2.12. Allocating a Slab Object

New objects may be obtained by invoking the `kmem_cache_alloc( )` function. The parameter `cache`

points to the cache descriptor from which the new free object must be obtained, while the parameter `flag` represents the flags to be passed to the zoned page frame allocator functions, should all slabs of the cache be full.

The function is essentially equivalent to the following:

```
void * kmem_cache_alloc(kmem_cache_t *cachep, int flags)
{
    unsigned long save_flags;
    void *objp;
    struct array_cache *ac;

    local_irq_save(save_flags);
    ac = cachep->array[smp_processor_id()];
    if (ac->avail) {
        ac->touched = 1;
        objp = ((void **) (ac+1)) [--ac->avail];
    } else
        objp = cache_alloc_refill(cachep, flags);
    local_irq_restore(save_flags);
    return objp;
}
```

The function tries first to retrieve a free object from the local cache. If there are free objects, the `avail` field contains the index in the local cache of the entry that points to the last freed object. Because the local cache array is stored right after the `ac` descriptor, `((void**) (ac+1)) [--ac->avail]` gets the address of that free object and decreases the value of `ac->avail`. The `cache_alloc_refill( )` function is invoked to repopulate the local cache and get a free object when there are no free objects in the local cache.

The `cache_alloc_refill( )` function essentially performs the following steps:

1. Stores in the `ac` local variable the address of the local cache descriptor:

```
ac = cachep->array[smp_processor_id()];
```

2. Gets the `cachep->spinlock`.
3. If the slab cache includes a shared local cache, and if the shared local cache includes some free objects, it refills the CPU's local cache by moving up to `ac->batchcount` pointers from the shared local cache. Then, it jumps to step 6.
4. Tries to fill the local cache with up to `ac->batchcount` pointers to free objects included in the slabs of the cache:

- a. Looks in the `slabs_partial` and `slabs_free` lists of the cache descriptor, and gets the address `slabp` of a slab descriptor whose corresponding slab is either partially filled or empty. If no such descriptor exists, the function goes to step 5.
- b. For each free object in the slab, the function increases the `inuse` field of the slab descriptor, inserts the object's address in the local cache, and updates the `free` field so that it stores the index of the next free object in the slab:

```
slabp->inuse++;
((void**) (ac+1)) [ac->avail++] =
    slabp->s_mem + slabp->free * cachep->obj_size;
slabp->free = ((kmem_bufctl_t*) (slabp+1)) [slabp->free];
```

- c. Inserts, if necessary, the depleted slab in the proper list, either the `slab_full` or the `slab_partial` list.
- 5. At this point, the number of pointers added to the local cache is stored in the `ac->avail` field: the function decreases the `free_objects` field of the `kmem_list3` structure of the same amount to specify that the objects are no longer free.
- 6. Releases the `cachep->spinlock`.
- 7. If the `ac->avail` field is now greater than 0 (some cache refilling took place), it sets the `ac->touched` field to 1 and returns the free object pointer that was last inserted in the local cache:

```
return ((void**) (ac+1)) [--ac->avail];
```

- 8. Otherwise, no cache refilling took place: invokes `cache_grow()` to get a new slab, and thus new free objects.
- 9. If `cache_grow()` fails, it returns NULL; otherwise it goes back to step 1 to repeat the procedure.

8.2.13. Freeing a Slab Object

The `kmem_cache_free()` function releases an object previously allocated by the slab allocator to some kernel function. Its parameters are `cachep`, the address of the cache descriptor, and `objp`, the address of the object to be released:

```
void kmem_cache_free(kmem_cache_t *cachep, void *objp)
{
    unsigned long flags;
    struct array_cache *ac;

    local_irq_save(flags);
    ac = cachep->array[smp_processor_id()];
    if (ac->avail == ac->limit)
        cache_flusharray(cachep, ac);
    ((void**) (ac+1)) [ac->avail++] = objp;
    local_irq_restore(flags);
}
```

The function checks first whether the local cache has room for an additional pointer to a free object. If so, the pointer is added to the local cache and the function returns. Otherwise it first invokes `cache_flusharray()` to deplete the local cache and then adds the pointer to the local cache.

The `cache_flusharray()` function performs the following operations:

- 1. Acquires the `cachep->spinlock` spin lock.
- 2. If the slab cache includes a shared local cache, and if the shared local cache is not already full, it refills the shared local cache by moving up to `ac->batchcount` pointers from the CPU's local cache. Then, it jumps to step 4.
- 3. Invokes the `free_block()` function to give back to the slab allocator up to `ac->batchcount` objects currently included in the local cache. For each object at address `objp`, the function executes the following steps:
 - a. Increases the `lists.free_objects` field of the cache descriptor.
 - b. Determines the address of the slab descriptor containing the object:

```
slabp = (struct slab *) (virt_to_page(objp)->lru.prev);
```

(Remember that the `lru.prev` field of the descriptor of the slab page points to the corresponding slab descriptor.)

- c. Removes the slab descriptor from its slab cache list (either `cachep->lists.slabs_partial` or `cachep->lists.slabs_full`).
- d. Computes the index of the object inside the slab:

```
objnr = (objp - slabp->s_mem) / cachep->objsize;
```

- e. Stores in the object descriptor the current value of the `slabp->free`, and puts in `slabp->free` the index of the object (the last released object will be the first object to be allocated again):

```
((kmem_bufctl_t *) (slabp+1))[objnr] = slabp->free;
slabp->free = objnr;
```

- f. Decreases the `slabp->inuse` field.
- g. If `slabp->inuse` is equal to zero all objects in the slab are free and the number of free objects in the whole slab cache (`cachep->lists.free_objects`) is greater than the limit stored in the `cachep->free_limit` field, then the function releases the slab's page frame(s) to the zoned page frame allocator:

```
cachep->lists.free_objects -= cachep->num;
slab_destroy(cachep, slabp);
```

The value stored in the `cachep->free_limit` field is usually equal to `cachep->num + (1+N) x cachep->batchcount`, where `N` denotes the number of CPUs of the system.

- h. Otherwise, if `slab->inuse` is equal to zero but the number of free objects in the whole slab cache is less than `cachep->free_limit`, it inserts the slab descriptor in the `cachep->lists.slabs_free` list.
 - i. Finally, if `slab->inuse` is greater than zero, the slab is partially filled, so the function inserts the slab descriptor in the `cachep->lists.slabs_partial` list.
4. Releases the `cachep->spinlock` spin lock.
 5. Updates the `avail` field of the local cache descriptor by subtracting the number of objects moved to the shared local cache or released to the slab allocator.
 6. Moves all valid pointers in the local cache at the beginning of the local cache's array. This step is necessary because the first object pointers have been removed from the local cache, thus the remaining ones must be moved up.

8.2.14. General Purpose Objects

As stated earlier in the section "[The Buddy System Algorithm](#)," infrequent requests for memory areas are handled through a group of general caches whose objects have geometrically distributed sizes ranging from a minimum of 32 to a maximum of 131,072 bytes.

Objects of this type are obtained by invoking the `kmalloc( )` function, which is essentially equivalent to the following code fragment:

```
void * kmalloc(size_t size, int flags)
{
    struct cache_sizes *csizep = malloc_sizes;
    kmem_cache_t * cachep;
    for (; csizep->cs_size; csizep++) {
        if (size > csizep->cs_size)
            continue;
        if (flags & __GFP_DMA)
            cachep = csizep->cs_dmacachep;
        else
            cachep = csizep->cs_cachep;
        return kmem_cache_alloc(cachep, flags);
    }
    return NULL;
}
```

The function uses the `malloc_sizes` table to locate the nearest power-of-2 size to the requested size. It then calls `kmem_cache_alloc( )` to allocate the object, passing to it either the cache descriptor for the page frames usable for ISA DMA or the cache descriptor for the "normal" page frames, depending on whether the caller specified the `__GFP_DMA` flag.

Objects obtained by invoking `kmalloc( )` can be released by calling `kfree( )`:

```
void kfree(const void *objp)
{
    kmem_cache_t * c;
    unsigned long flags;
    if (!objp)
        return;
    local_irq_save(flags);
    c = (kmem_cache_t *) (virt_to_page(objp)->lru.next);
    kmem_cache_free(c, (void *)objp);
    local_irq_restore(flags);
}
```

The proper cache descriptor is identified by reading the `lru.next` subfield of the descriptor of the first page frame containing the memory area. The memory area is released by invoking `kmem_cache_free( )`.

8.2.15. Memory Pools

Memory pools are a new feature of Linux 2.6. Basically, a memory pool allows a kernel component such as the block device subsystem to allocate some dynamic memory to be used only in low-on-memory emergencies.

Memory pools should not be confused with the reserved page frames described in the earlier section "[The Pool of Reserved Page Frames](#)." In fact, those page frames can be used only to satisfy atomic memory allocation requests issued by interrupt handlers or inside critical regions. Instead, a memory pool is a reserve of dynamic memory that can be used only by a specific kernel component, namely the "owner" of the pool. The owner does not normally use the reserve; however, if dynamic memory becomes so scarce that all usual

memory allocation requests are doomed to fail, the kernel component can invoke, as a last resort, special memory pool functions that dip in the reserve and get the memory needed. Thus, creating a memory pool is similar to keeping a reserve of canned foods on hand and using a can opener only when no fresh food is available.

Often, a memory pool is stacked over the slab allocator that is, it is used to keep a reserve of slab objects. Generally speaking, however, a memory pool can be used to allocate every kind of dynamic memory, from whole page frames to small memory areas allocated with `kmalloc()`. Therefore, we will generically refer to the memory units handled by a memory pool as "memory elements."

A memory pool is described by a `mempool_t` object, whose fields are shown in [Table 8-12](#).

Table 8-12. The fields of the `mempool_t` object

Type

Name

Description

`spinlock_t`

`lock`

Spin lock protecting the object fields

`int`

`min_nr`

Maximum number of elements in the memory pool

`int`

`curr_nr`

Current number of elements in the memory pool

`void **`

`elements`

Pointer to an array of pointers to the reserved elements

`void *`

`pool_data`

Private data available to the pool's owner

`mempool_alloc_t *`

`alloc`

Method to allocate an element

```
mempool_free_t *
free
```

Method to free an element

```
wait_queue_head_t
wait
```

Wait queue used when the memory pool is empty

The `min_nr` field stores the initial number of elements in the memory pool. In other words, the value stored in this field represents the number of memory elements that the owner of the memory pool is sure to obtain from the memory allocator. The `curr_nr` field, which is always lower than or equal to `min_nr`, stores the number of memory elements currently included in the memory pool. The memory elements themselves are referenced by an array of pointers, whose address is stored in the `elements` field.

The `alloc` and `free` methods interface with the underlying memory allocator to get and release a memory element, respectively. Both methods may be custom functions provided by the kernel component that owns the memory pool.

When the memory elements are slab objects, the `alloc` and `free` methods are commonly implemented by the `mempool_alloc_slab()` and `mempool_free_slab()` functions, which just invoke the `kmem_cache_alloc()` and `kmem_cache_free()` functions, respectively. In this case, the `pool_data` field of the `mempool_t` object stores the address of the slab cache descriptor.

The `mempool_create()` function creates a new memory pool; it receives the number of memory elements `min_nr`, the addresses of the functions that implement the `alloc` and `free` methods, and an optional value for the `pool_data` field. The function allocates memory for the `mempool_t` object and the array of pointers to the memory elements, then repeatedly invokes the `alloc` method to get the `min_nr` memory elements. Conversely, the `mempool_destroy()` function releases all memory elements in the pool, then releases the array of elements and the `mempool_t` object themselves.

To allocate an element from a memory pool, the kernel invokes the `mempool_alloc()` function, passing to it the address of the `mempool_t` object and the memory allocation flags (see [Table 8-5](#) and [Table 8-6](#) earlier in this chapter). Essentially, the function tries to allocate a memory element from the underlying memory allocator by invoking the `alloc` method, according to the memory allocation flags specified as parameters. If the allocation succeeds, the function returns the memory element obtained, without touching the memory pool. Otherwise, if the allocation fails, the memory element is taken from the memory pool. Of course, too many allocations in a low-on-memory condition can exhaust the memory pool: in this case, if the `__GFP_WAIT` flag is not set, `mempool_alloc()` blocks the current process until a memory element is released to the memory pool.

Conversely, to release an element to a memory pool, the kernel invokes the `mempool_free()` function. If the memory pool is not full (`curr_min` is smaller than `min_nr`), the function adds the element to the memory pool. Otherwise, `mempool_free()` invokes the `free` method to release the element to the underlying memory allocator.

8.3. Noncontiguous Memory Area Management

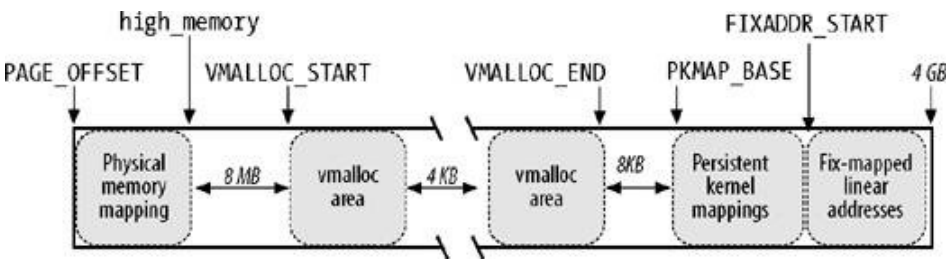
We already know that it is preferable to map memory areas into sets of contiguous page frames, thus making better use of the cache and achieving lower average memory access times. Nevertheless, if the requests for memory areas are infrequent, it makes sense to consider an allocation scheme based on noncontiguous page frames accessed through contiguous linear addresses. The main advantage of this schema is to avoid external fragmentation, while the disadvantage is that it is necessary to fiddle with the kernel Page Tables. Clearly, the size of a noncontiguous memory area must be a multiple of 4,096. Linux uses noncontiguous memory areas in several ways for instance, to allocate data structures for active swap areas (see the section "[Activating and Deactivating a Swap Area](#)" in [Chapter 17](#)), to allocate space for a module (see [Appendix B](#)), or to allocate buffers to some I/O drivers. Furthermore, noncontiguous memory areas provide yet another way to make use of high memory page frames (see the later section "[Allocating a Noncontiguous Memory Area](#)").

8.3.1. Linear Addresses of Noncontiguous Memory Areas

To find a free range of linear addresses, we can look in the area starting from `PAGE_OFFSET` (usually `0xc0000000`, the beginning of the fourth gigabyte). [Figure 8-7](#) shows how the fourth gigabyte linear addresses are used:

- The beginning of the area includes the linear addresses that map the first 896 MB of RAM (see the section "[Process Page Tables](#)" in [Chapter 2](#)); the linear address that corresponds to the end of the directly mapped physical memory is stored in the `high_memory` variable.
- The end of the area contains the fix-mapped linear addresses (see the section "[Fix-Mapped Linear Addresses](#)" in [Chapter 2](#)).
- Starting from `PKMAP_BASE` we find the linear addresses used for the persistent kernel mapping of high-memory page frames (see the section "[Kernel Mappings of High-Memory Page Frames](#)" earlier in this chapter).
- The remaining linear addresses can be used for noncontiguous memory areas. A safety interval of size 8 MB (macro `VMALLOC_OFFSET`) is inserted between the end of the physical memory mapping and the first memory area; its purpose is to "capture" out-of-bounds memory accesses. For the same reason, additional safety intervals of size 4 KB are inserted to separate noncontiguous memory areas.

Figure 8-7. The linear address interval starting from `PAGE_OFFSET`



The VMALLOC_START macro defines the starting address of the linear space reserved for noncontiguous memory areas, while VMALLOC_END defines its ending address.

8.3.2. Descriptors of Noncontiguous Memory Areas

Each noncontiguous memory area is associated with a descriptor of type vm_struct, whose fields are listed in Table 8-13.

Table 8-13. The fields of the vm_struct descriptor

Type	Name	Description
void *	addr	Linear address of the first memory cell of the area
unsigned long	size	Size of the area plus 4,096 (inter-area safety interval)
unsigned long	flags	Type of memory mapped by the noncontiguous memory area
struct page **	pages	Pointer to array of nr_pages pointers to page descriptors
unsigned int	nr_pages	Number of pages filled by the area
unsigned long	phys_addrmap	Set to 0 unless the area has been created to map the I/O shared memory of a hardware device
struct vm_struct *	next	

These descriptors are inserted in a simple list by means of the `next` field; the address of the first element of the list is stored in the `vm_list` variable. Accesses to this list are protected by means of the `vm_list_lock` read/write spin lock. The `flags` field identifies the type of memory mapped by the area: `VM_ALLOC` for pages obtained by means of `vmalloc()`, `VM_MAP` for already allocated pages mapped by means of `vmap()` (see the next section), and `VM_IOREMAP` for on-board memory of hardware devices mapped by means of `ioremap()` (see [Chapter 13](#)).

The `get_vm_area()` function looks for a free range of linear addresses between `VMALLOC_START` and `VMALLOC_END`. This function acts on two parameters: the size (`size`) in bytes of the memory region to be created, and a flag (`flag`) specifying the type of region (see above). The steps performed are the following:

1. Invokes `kmalloc()` to obtain a memory area for the new descriptor of type `vm_struct`.
2. Gets the `vm_list_lock` lock for writing and scans the list of descriptors of type `vm_struct` looking for a free range of linear addresses that includes at least `size + 4096` addresses (4096 is the size of the safety interval between the memory areas).
3. If such an interval exists, the function initializes the fields of the descriptor, releases the `vm_list_lock` lock, and terminates by returning the initial address of the noncontiguous memory area.
4. Otherwise, `get_vm_area()` releases the descriptor obtained previously, releases the `vm_list_lock` lock, and returns `NULL`.

8.3.3. Allocating a Noncontiguous Memory Area

The `vmalloc()` function allocates a noncontiguous memory area to the kernel. The parameter `size` denotes the size of the requested area. If the function is able to satisfy the request, it then returns the initial linear address of the new area; otherwise, it returns a `NULL` pointer:

```
void * vmalloc(unsigned long size)
{
    struct vm_struct *area;
    struct page **pages;
    unsigned int array_size, i;
    size = (size + PAGE_SIZE - 1) & PAGE_MASK;
    area = get_vm_area(size, VM_ALLOC);
    if (!area)
        return NULL;
    area->nr_pages = size >> PAGE_SHIFT;
    array_size = (area->nr_pages * sizeof(struct page *));
    area->pages = pages = kmalloc(array_size, GFP_KERNEL);
    if (!area->pages) {
        remove_vm_area(area->addr);
        kfree(area);
        return NULL;
    }
    memset(area->pages, 0, array_size);
    for (i=0; i<area->nr_pages; i++) {
        area->pages[i] = alloc_page(GFP_KERNEL|__GFP_HIGHMEM);
        if (!area->pages[i]) {
            area->nr_pages = i;
fail:      vfree(area->addr);
            return NULL;
        }
    }
}
```

```

    }
}
if (map_vm_area(area, __pgprot(0x63), &pages))
    goto fail;
return area->addr;
}

```

The function starts by rounding up the value of the `size` parameter to a multiple of 4,096 (the page frame size). Then `vmalloc( )` invokes `get_vm_area( )`, which creates a new descriptor and returns the linear addresses assigned to the memory area. The `flags` field of the descriptor is initialized with the `VM_ALLOC` flag, which means that the noncontiguous page frames will be mapped into a linear address range by means of the `vmalloc( )` function. Then the `vmalloc( )` function invokes `kmalloc( )` to request a group of contiguous page frames large enough to contain an array of page descriptor pointers. The `memset( )` function is invoked to set all these pointers to `NULL`. Next the `alloc_page( )` function is called repeatedly, once for each of the `nr_pages` of the region, to allocate a page frame and store the address of the corresponding page descriptor in the `area->pages` array. Observe that using the `area->pages` array is necessary because the page frames could belong to the `ZONE_HIGHMEM` memory zone, thus right now they are not necessarily mapped to a linear address.

Now comes the tricky part. Up to this point, a fresh interval of contiguous linear addresses has been obtained and a group of noncontiguous page frames has been allocated to map these linear addresses. The last crucial step consists of fiddling with the page table entries used by the kernel to indicate that each page frame allocated to the noncontiguous memory area is now associated with a linear address included in the interval of contiguous linear addresses yielded by `vmalloc( )`. This is what `map_vm_area( )` does.

The `map_vm_area( )` function uses three parameters:

`area`

The pointer to the `vm_struct` descriptor of the area.

`prot`

The protection bits of the allocated page frames. It is always set to `0x63`, which corresponds to Present, Accessed, Read/Write, and Dirty.

`pages`

The address of a variable pointing to an array of pointers to page descriptors (thus, `struct page ***` is used as the data type!).

The function starts by assigning the linear addresses of the start and end of the area to the `address` and `end` local variables, respectively:

```

address = area->addr;
end = address + (area->size - PAGE_SIZE);

```

Remember that `area->size` stores the actual size of the area plus the 4 KB inter-area safety interval. The function then uses the `pgd_offset_k` macro to derive the entry in the master kernel Page Global Directory related to the initial linear address of the area; it then acquires the kernel Page Table spin lock:

```
pgd = pgd_offset_k(address);
spin_lock(&init_mm.page_table_lock);
```

The function then executes the following cycle:

```
int ret = 0;
for (i = pgd_index(address); i < pgd_index(end-1); i++) {
    pud_t *pud = pud_alloc(&init_mm, pgd, address);
    ret = -ENOMEM;
    if (!pud)
        break;
    next = (address + PGDIR_SIZE) & PGDIR_MASK;
    if (next < address || next > end)
        next = end;
    if (map_area_pud(pud, address, next, prot, pages))
        break;
    address = next;
    pgd++;
    ret = 0;
}
spin_unlock(&init_mm.page_table_lock);
flush_cache_vmap((unsigned long)area->addr, end);
return ret;
```

In each cycle, it first invokes `pud_alloc( )` to create a Page Upper Directory for the new area and writes its physical address in the right entry of the kernel Page Global Directory. It then calls `map_area_pud( )` to allocate all the page tables associated with the new Page Upper Directory. It adds the size of the range of linear addresses spanned by a single Page Upper Directory the constant 2^{30} if PAE is enabled, 2^{22} otherwise to the current value of `address`, and it increases the pointer `pgd` to the Page Global Directory.

The cycle is repeated until all Page Table entries referring to the noncontiguous memory area are set up.

The `map_area_pud( )` function executes a similar cycle for all the page tables that a Page Upper Directory points to:

```
do {
    pmd_t *pmd = pmd_alloc(&init_mm, pud, address);
    if (!pmd)
        return -ENOMEM;
    if (map_area_pmd(pmd, address, end-address, prot, pages))
        return -ENOMEM;
    address = (address + PUD_SIZE) & PUD_MASK;
    pud++;
} while (address < end);
```

The `map_area_pmd( )` function executes a similar cycle for all the Page Tables that a Page Middle Directory points to:

```
do {
    pte_t *pte = pte_alloc_kernel(&init_mm, pmd, address);
    if (!pte)
        return -ENOMEM;
    if (map_area_pte(pte, address, end-address, prot, pages))
        return -ENOMEM;
    address = (address + PMD_SIZE) & PMD_MASK;
```

```

    pmd++;
} while (address < end);

```

The `pte_alloc_kernel( )` function (see the section "[Page Table Handling](#)" in [Chapter 2](#)) allocates a new Page Table and updates the corresponding entry in the Page Middle Directory. Next, `map_area_pte( )` allocates all the page frames corresponding to the entries in the Page Table. The value of `address` is increased by 2^{22} the size of the linear address interval spanned by a single Page Table and the cycle is repeated.

The main cycle of `map_area_pte( )` is:

```

do {
    struct page * page = **pages;
    set_pte(pte, mk_pte(page, prot));
    address += PAGE_SIZE;
    pte++;
    (*pages)++;
} while (address < end);

```

The page descriptor address `page` of the page frame to be mapped is read from the array's entry pointed to by the variable at address `pages`. The physical address of the new page frame is written into the Page Table by the `set_pte` and `mk_pte` macros. The cycle is repeated after adding the constant 4,096 (the length of a page frame) to `address`.

Notice that the Page Tables of the current process are not touched by `map_vm_area( )`. Therefore, when a process in Kernel Mode accesses the noncontiguous memory area, a Page Fault occurs, because the entries in the process's Page Tables corresponding to the area are null. However, the Page Fault handler checks the faulty linear address against the master kernel Page Tables (which are `init_mm.pgd` Page Global Directory and its child page tables; see the section "[Kernel Page Tables](#)" in [Chapter 2](#)). Once the handler discovers that a master kernel Page Table includes a non-null entry for the address, it copies its value into the corresponding process's Page Table entry and resumes normal execution of the process. This mechanism is described in the section "[Page Fault Exception Handler](#)" in [Chapter 9](#).

Beside the `vmalloc( )` function, a noncontiguous memory area can be allocated by the `vmalloc_32( )` function, which is very similar to `vmalloc( )` but only allocates page frames from the `ZONE_NORMAL` and `ZONE_DMA` memory zones.

Linux 2.6 also features a `vmap( )` function, which maps page frames already allocated in a noncontiguous memory area: essentially, this function receives as its parameter an array of pointers to page descriptors, invokes `get_vm_area( )` to get a new `vm_struct` descriptor, and then invokes `map_vm_area( )` to map the page frames. The function is thus similar to `vmalloc( )`, but it does not allocate page frames.

8.3.4. Releasing a Noncontiguous Memory Area

The `vfree( )` function releases noncontiguous memory areas created by `vmalloc( )` or `vmalloc_32( )`, while the `vunmap( )` function releases memory areas created by `vmap( )`. Both functions have one parameter the address of the initial linear address of the area to be released; they both rely on the `_vunmap( )` function to do the real work.

The `__vunmap( )` function receives two parameters: the address `addr` of the initial linear address of the area to be released, and the flag `deallocate_pages`, which is set if the page frames mapped in the area should be released to the zoned page frame allocator (`vfree( )`'s invocation), and cleared otherwise (`vunmap( )`'s invocation). The function performs the following operations:

1. Invokes the `remove_vm_area( )` function to get the address `area` of the `vm_struct` descriptor and to clear the kernel's page table entries corresponding to the linear address in the noncontiguous memory area.
2. If the `deallocate_pages` flag is set, it scans the `area->pages` array of pointers to the page descriptor; for each element of the array, invokes the `__free_page( )` function to release the page frame to the zoned page frame allocator. Moreover, executes `kfree(area->pages)` to release the array itself.
3. Invokes `kfree(area)` to release the `vm_struct` descriptor.

The `remove_vm_area( )` function performs the following cycle:

```
write_lock(&vmlist_lock);
for (p = &vmlist ; (tmp = *p) ; p = &tmp->next) {
    if (tmp->addr == addr) {
        unmap_vm_area(tmp);
        *p = tmp->next;
        break;
    }
}
write_unlock(&vmlist_lock);
return tmp;
```

The area itself is released by invoking `unmap_vm_area( )`. This function acts on a single parameter, namely a pointer `area` to the `vm_struct` descriptor of the area. It executes the following cycle to reverse the actions performed by `map_vm_area( )`:

```
address = area->addr;
end = address + area->size;
pgd = pgd_offset_k(address);
for (i = pgd_index(address); i <= pgd_index(end-1); i++) {
    next = (address + PGDIR_SIZE) & PGDIR_MASK;
    if (next <= address || next > end)
        next = end;
    unmap_area_pud(pgd, address, next - address);
    address = next;
    pgd++;
}
```

In turn, `unmap_area_pud( )` reverses the actions of `map_area_pud( )` in the cycle:

```
do {
    unmap_area_pmd(pud, address, end-address);
    address = (address + PUD_SIZE) & PUD_MASK;
    pud++;
} while (address && (address < end));
```

The `unmap_area_pmd( )` function reverses the actions of `map_area_pmd( )` in the cycle:

```
do {
```

```

    unmap_area_pte(pmd, address, end-address);
    address = (address + PMD_SIZE) & PMD_MASK;
    pmd++;
} while (address < end);

```

Finally, `unmap_area_pte( )` reverses the actions of `map_area_pte( )` in the cycle:

```

do {
    pte_t page = ptep_get_and_clear(pte);
    address += PAGE_SIZE;
    pte++;
    if (!pte_none(page) && !pte_present(page))
        printk("Whee... Swapped out page in kernel page table\n");
} while (address < end);

```

In every iteration of the cycle, the page table entry pointed to by `pte` is set to 0 by the `ptep_get_and_clear` macro.

As for `vmalloc( )`, the kernel modifies the entries of the master kernel Page Global Directory and its child page tables (see the section "[Kernel Page Tables](#)" in [Chapter 2](#)), but it leaves unchanged the entries of the process page tables mapping the fourth gigabyte. This is fine because the kernel never reclaims Page Upper Directories, Page Middle Directories, and Page Tables rooted at the master kernel Page Global Directory.

For instance, suppose that a process in Kernel Mode accessed a noncontiguous memory area that later got released. The process's Page Global Directory entries are equal to the corresponding entries of the master kernel Page Global Directory, thanks to the mechanism explained in the section "[Page Fault Exception Handler](#)" in [Chapter 9](#); they point to the same Page Upper Directories, Page Middle Directories, and Page Tables. The `unmap_area_pte( )` function clears only the entries of the page tables (without reclaiming the page tables themselves). Further accesses of the process to the released noncontiguous memory area will trigger Page Faults because of the null page table entries. However, the handler will consider such accesses a bug, because the master kernel page tables do not include valid entries.